Background-

Integrate the <Orb /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: Orb
Variant: JavaScript + CSS

Usage Example
```jsx
import Orb from './Orb';

<div style={{ width: '100%', height: '600px', position: 'relative' }}>
  <Orb
    colors={['#9c43fe', '#4cc2e9', '#101499']}
    hoverIntensity={1}
    trail={1}
    rotateOnHover
    followCursor
  />
</div>
```

Props
Prop	Type	Default	Description
colors	string[]	['#9c43fe', '#4cc2e9', '#101499']	Two colors that swirl around the ring, then the deep color of the glow behind it.
hue	number	0	Shifts the hue of every color, in degrees.
backgroundColor	string	'#000000'	Color the orb sits on. Light colors switch it to a soft, filled style.
size	number	1	Size of the orb relative to the container.
glow	number	1	How soft and wide the glow around the ring is.
bleed	number	1	How far the glow bleeds into the inside of the orb. 0 keeps a tight ring.
wobble	number	1	How much the ring breathes and warps at rest.
highlight	number	1	Brightness of the light that orbits the ring.
trail	number	1	Length of the glowing tail behind the orbiting light. 0 removes it.
tiltX	number	0	Tilts the ring back in perspective, in degrees. Positive values push the top away.
tiltY	number	0	Turns the ring sideways in perspective, in degrees.
speed	number	1	Speed of the swirl, the orbiting light and the breathing.
hoverIntensity	number	1	How far the ring reaches toward the cursor and how brightly it lights up there.
rotateOnHover	boolean	true	Slowly turns the orb while it is hovered.
forceHoverState	boolean	false	Keeps the hover effect on without hovering.
followCursor	boolean	true	The orbiting light swings toward the cursor while the orb is hovered.
clickPulse	boolean	true	Clicking the orb makes it pulse and swirl.
intro	boolean	true	Powers the orb on with a spring and a swirl.
opacity	number	1	Opacity of the whole effect.
paused	boolean	false	Freezes the animation.
dpr	number	auto	Maximum device pixel ratio, capped at 2.
className	string	''	Extra class names for the container.
style	CSSProperties	-	Inline styles for the container.

Full Component Source
```jsx
'use client';

import { useEffect, useRef } from 'react';
import './Orb.css';

const INTRO_SECONDS = 1.1;
const FOCAL = 3.2;

const VERTEX = `#version 300 es
in vec2 aPosition;
void main() {
  gl_Position = vec4(aPosition, 0.0, 1.0);
}`;

const FRAGMENT = `#version 300 es
precision highp float;
```

```
uniform vec2 uResolution;
uniform float uTime;
uniform float uRotation;
uniform vec2 uReach;
uniform float uNear;
uniform float uAwake;
uniform vec3 uBackground;
uniform vec3 uColor1;
uniform vec3 uColor2;
uniform vec3 uColor3;
uniform float uScale;
uniform float uWobble;
uniform float uGlow;
uniform float uBleed;
uniform float uHighlight;
uniform float uSwirl;
uniform float uOrbit;
uniform vec2 uEcho;
uniform float uReveal;
uniform float uOpacity;
uniform float uTrail;
uniform vec2 uTilt;
out vec4 outColor;

vec3 hash33(vec3 p3) {
  p3 = fract(p3 * vec3(0.1031, 0.11369, 0.13787));
  p3 += dot(p3, p3.yxz + 19.19);
  return -1.0 + 2.0 * fract(vec3(p3.x + p3.y, p3.x + p3.z, p3.y + p3.z) * p3.zyx);
}

float snoise3(vec3 p) {
  const float K1 = 0.333333333;
  const float K2 = 0.166666667;
  vec3 i = floor(p + (p.x + p.y + p.z) * K1);
  vec3 d0 = p - (i - (i.x + i.y + i.z) * K2);
  vec3 e = step(vec3(0.0), d0 - d0.yzx);
  vec3 i1 = e * (1.0 - e.zxy);
  vec3 i2 = 1.0 - e.zxy * (1.0 - e);
  vec3 d1 = d0 - (i1 - K2);
  vec3 d2 = d0 - (i2 - K1);
  vec3 d3 = d0 - 0.5;
  vec4 h = max(0.6 - vec4(dot(d0, d0), dot(d1, d1), dot(d2, d2), dot(d3, d3)), 0.0);
  vec4 n = h * h * h * h * vec4(dot(d0, hash33(i)), dot(d1, hash33(i + i1)), dot(d2, hash33(i + i2)), dot(d3, hash33(i + 1.0)));
  return dot(vec4(31.316), n);
}

float light1(float intensity, float attenuation, float dist) {
  return intensity / (1.0 + dist * attenuation);
}

float light2(float intensity, float attenuation, float dist) {
  return intensity / (1.0 + dist * dist * attenuation);
}

vec4 draw(vec2 uv) {
  float rawLen = length(uv);
  vec2 dir = rawLen > 0.0001 ? uv / rawLen : vec2(1.0, 0.0);
  float reachAmount = length(uReach);
  vec2 reachDir = reachAmount > 0.00001 ? uReach / reachAmount : vec2(1.0, 0.0);
  float facing = exp((dot(dir, reachDir) - 1.0) * 3.2);
  float len = max(rawLen - reachAmount * facing * smoothstep(0.25, 0.65, rawLen), 0.0);
  uv = dir * len;
  float ang = atan(dir.y, dir.x);
  float invLen = len > 0.0 ? 1.0 / len : 0.0;
  float near = facing * uNear;
  float bgLuminance = dot(uBackground, vec3(0.299, 0.587, 0.114));

  float n0 = snoise3(vec3(uv * 0.65, uTime * 0.5)) * 0.5 + 0.5;
  float r0 = mix(0.8 - 0.04 * uWobble, 0.8 + 0.04 * uWobble, n0);
  float d0 = distance(uv, (r0 * invLen) * uv);
  float v0 = light1(1.0, 10.0 / uGlow, d0);
  v0 *= smoothstep(r0 * 1.05, r0, len);
  float reach = max(uBleed, 0.04);
  float innerFade = smoothstep(r0 * (1.0 - 0.2 * reach), r0 * (1.0 - 0.05 * reach), len);
```

```
  v0 *= mix(innerFade, 1.0, bgLuminance * 0.7 * min(uBleed, 1.0));
  float body = smoothstep(0.8 - 0.2 * reach, 0.8, len);
  float cl = cos(ang + uSwirl) * 0.5 + 0.5;

  vec2 pos = vec2(cos(uOrbit), sin(uOrbit)) * r0;
  float d = distance(uv, pos);
  vec2 head = vec2(cos(uOrbit), sin(uOrbit));
  float lag = atan(head.x * dir.y - head.y * dir.x, dot(dir, head));
  float trail = uTrail > 0.0 ? smoothstep(0.0, 0.35, lag) * exp(-max(lag, 0.0) * 2.6 / uTrail) : 0.0;
  float v1 = (light2(1.5, 5.0, d) + trail * 0.6) * uHighlight;
  v1 *= light1(1.0, 50.0, d0);
  float band = v0 * smoothstep(1.0, mix(0.6, 1.0, n0 * 0.5), len) * body;
  float heat = pow(clamp(band / 0.82, 0.0, 1.0), 3.0);
  float bloom = light2(1.0, 160.0 / uGlow, max(len - r0, 0.0)) * smoothstep(0.4, 0.0, len - r0) * step(r0, len);

  float v2 = smoothstep(1.0, mix(0.6, 1.0, n0 * 0.5), len);
  float v3 = body;

  vec3 colBase = mix(uColor1, uColor2, cl);
  float fadeAmount = mix(1.0, 0.1, bgLuminance);

  float echoD = abs(len - r0 - uEcho.x);
  float echo = light1(1.0, 60.0, echoD) * smoothstep(0.12, 0.0, echoD) * uEcho.y;

  vec3 darkCol = mix(uColor3, colBase, v0);
  darkCol = (darkCol + v1) * v2 * v3 * (1.0 + uAwake + near * 0.35) + mix(colBase, vec3(1.0), 0.25) * heat * (0.28 + near * 0.3);
  darkCol = clamp(darkCol + colBase * (bloom * 0.1 + echo), 0.0, 1.0);

  vec3 lightCol = (colBase + v1) * mix(1.0, v2 * v3, fadeAmount);
  lightCol = mix(uBackground, lightCol, clamp(v0 + echo, 0.0, 1.0));
  lightCol = mix(lightCol, colBase, clamp(v0, 0.0, 1.0) * clamp(near * 0.4 + uAwake, 0.0, 0.6));
  lightCol = clamp(mix(lightCol, vec3(1.0), clamp(v1 * 0.35, 0.0, 0.6)), 0.0, 1.0);

  vec3 finalCol = mix(darkCol, lightCol, bgLuminance);
  float alpha = max(max(finalCol.r, finalCol.g), finalCol.b);
  return vec4(finalCol / (alpha + 1e-5), alpha);
}

vec2 toPlane(vec2 p, out float hit) {
  float cx = cos(uTilt.x);
  float sx = sin(uTilt.x);
  float cy = cos(uTilt.y);
  float sy = sin(uTilt.y);
  vec3 ex = vec3(cy, 0.0, -sy);
  vec3 ey = vec3(sy * sx, cx, cy * sx);
  vec3 normal = vec3(sy * cx, -sx, cy * cx);
  vec3 origin = vec3(0.0, 0.0, ${FOCAL.toFixed(1)});
  vec3 ray = vec3(p, -${FOCAL.toFixed(1)});
  float denom = dot(normal, ray);
  float t = -dot(normal, origin) / (abs(denom) > 0.00001 ? denom : -0.00001);
  hit = t > 0.0 ? 1.0 : 0.0;
  vec3 q = origin + ray * t;
  return vec2(dot(q, ex), dot(q, ey));
}

void main() {
  vec2 uv = (gl_FragCoord.xy - uResolution * 0.5) / min(uResolution.x, uResolution.y) * 2.0;
  uv *= uScale;
  float hit = 1.0;
  if (abs(uTilt.x) + abs(uTilt.y) > 0.0001) uv = toPlane(uv, hit);

  float s = sin(uRotation);
  float c = cos(uRotation);
  uv = vec2(c * uv.x - s * uv.y, s * uv.x + c * uv.y);

  vec4 col = draw(uv);
  float dither = (fract(sin(dot(gl_FragCoord.xy, vec2(12.9898, 78.233))) * 43758.5453) - 0.5) / 255.0;
  float alpha = clamp(col.a * hit * uReveal * uOpacity + dither, 0.0, 1.0);
  outColor = vec4(col.rgb * alpha * alpha, alpha);
}`;

const clamp = (value, min, max) => Math.min(max, Math.max(min, value));

const compile = (gl, type, source) => {
```

```
  const shader = gl.createShader(type);
  if (!shader) return null;
  gl.shaderSource(shader, source);
  gl.compileShader(shader);
  if (gl.getShaderParameter(shader, gl.COMPILE_STATUS)) return shader;
  gl.deleteShader(shader);
  return null;
};

const link = gl => {
  const vertex = compile(gl, gl.VERTEX_SHADER, VERTEX);
  const fragment = compile(gl, gl.FRAGMENT_SHADER, FRAGMENT);
  const program = gl.createProgram();
  if (!vertex || !fragment || !program) return null;
  gl.attachShader(program, vertex);
  gl.attachShader(program, fragment);
  gl.bindAttribLocation(program, 0, 'aPosition');
  gl.linkProgram(program);
  gl.deleteShader(vertex);
  gl.deleteShader(fragment);
  if (!gl.getProgramParameter(program, gl.LINK_STATUS)) {
    gl.deleteProgram(program);
    return null;
  }
  const uniforms = {};
  const count = gl.getProgramParameter(program, gl.ACTIVE_UNIFORMS);
  for (let i = 0; i < count; i++) {
    const info = gl.getActiveUniform(program, i);
    if (info) uniforms[info.name] = gl.getUniformLocation(program, info.name);
  }
  return { program, uniforms };
};

const Orb = ({
  colors = ['#9c43fe', '#4cc2e9', '#101499'],
  hue = 0,
  backgroundColor = '#000000',
  size = 1,
  glow = 1,
  bleed = 1,
  wobble = 1,
  highlight = 1,
  trail = 1,
  tiltX = 0,
  tiltY = 0,
  speed = 1,
  hoverIntensity = 1,
  rotateOnHover = true,
  forceHoverState = false,
  followCursor = true,
  clickPulse = true,
  intro = true,
  opacity = 1,
  paused = false,
  dpr,
  className = '',
  style
}) => {
  const containerRef = useRef(null);
  const canvasRef = useRef(null);
  const wakeRef = useRef(null);
  const palette = Array.isArray(colors) ? colors : [];
  const settings = {
    colors: [palette[0] ?? '#9c43fe', palette[1] ?? palette[0] ?? '#4cc2e9', palette[2] ?? '#101499'].map(String),
    hue,
    backgroundColor: String(backgroundColor),
    size: clamp(size, 0.1, 3),
    glow: clamp(glow, 0.1, 5),
    bleed: clamp(bleed, 0, 4),
    wobble: Math.max(0, wobble),
    highlight: Math.max(0, highlight),
    trail: Math.max(0, trail),
    tiltX: clamp(tiltX, -70, 70),
    tiltY: clamp(tiltY, -70, 70),
```

```
  speed,
  hoverIntensity: Math.max(0, hoverIntensity),
  rotateOnHover,
  forceHoverState,
  followCursor,
  clickPulse,
  intro,
  opacity: clamp(opacity, 0, 1),
  paused,
  dpr
};
const settingsRef = useRef(settings);
settingsRef.current = settings;

useEffect(() => {
  const container = containerRef.current;
  const canvas = canvasRef.current;
  const gl = canvas?.getContext('webgl2', {
    alpha: true,
    premultipliedAlpha: true,
    antialias: false,
    depth: false,
    stencil: false
  });
  if (!container || !canvas || !gl) return undefined;

  const program = link(gl);
  if (!program) return undefined;
  const u = program.uniforms;

  const buffer = gl.createBuffer();
  const vao = gl.createVertexArray();
  gl.bindVertexArray(vao);
  gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([-1, -1, 3, -1, -1, 3]), gl.STATIC_DRAW);
  gl.enableVertexAttribArray(0);
  gl.vertexAttribPointer(0, 2, gl.FLOAT, false, 0, 0);

  const probe = document.createElement('canvas').getContext('2d', { willReadFrequently: true });
  const reduce = window.matchMedia?.('(prefers-reduced-motion: reduce)').matches ?? false;
  const cache = new Map();
  const parse = value => {
    if (cache.has(value)) return cache.get(value);
    let rgb = [0, 0, 0];
    if (probe) {
      probe.clearRect(0, 0, 1, 1);
      probe.fillStyle = '#000000';
      probe.fillStyle = value;
      probe.fillRect(0, 0, 1, 1);
      const [r, g, b, a] = probe.getImageData(0, 0, 1, 1).data;
      rgb = a ? [r / 255, g / 255, b / 255] : [0, 0, 0];
    }
    cache.set(value, rgb);
    return rgb;
  };
  const shiftHue = (rgb, degrees) => {
    if (!degrees) return rgb;
    const angle = (degrees * Math.PI) / 180;
    const cos = Math.cos(angle);
    const sin = Math.sin(angle);
    const [r, g, b] = rgb;
    const y = 0.299 * r + 0.587 * g + 0.114 * b;
    const i = 0.596 * r - 0.274 * g - 0.322 * b;
    const q = 0.211 * r - 0.523 * g + 0.312 * b;
    const ni = i * cos - q * sin;
    const nq = i * sin + q * cos;
    return [y + 0.956 * ni + 0.621 * nq, y - 0.272 * ni - 0.647 * nq, y - 1.106 * ni + 1.703 * nq];
  };

  const view = { width: 1, height: 1 };
  const pointer = { x: 0, y: 0, inside: false };
  const state = {
    time: 0,
    swirl: 0,
```

```
  orbit: 0,
  echo: 1,
  rotation: 0,
  hover: 0,
  near: 0,
  reachX: 0,
  reachY: 0,
  reachSpeedX: 0,
  reachSpeedY: 0,
  pulse: 0,
  pulseSpeed: 0,
  kick: settingsRef.current.intro && !reduce ? 2.5 : 0,
  reveal: settingsRef.current.intro && !reduce ? 0 : 1
};
let raf = 0;
let last = 0;
let visible = true;

const toOrb = (x, y) => {
  const s = settingsRef.current;
  const side = Math.min(view.width, view.height);
  const scale = 1 / (s.size * (1 + state.pulse));
  let ux = ((x - view.width / 2) / side) * 2 * scale;
  let uy = ((view.height / 2 - y) / side) * 2 * scale;
  if (s.tiltX || s.tiltY) {
    const ax = (-s.tiltX * Math.PI) / 180;
    const ay = (s.tiltY * Math.PI) / 180;
    const [cx, sx, cy, sy] = [Math.cos(ax), Math.sin(ax), Math.cos(ay), Math.sin(ay)];
    const normal = [sy * cx, -sx, cy * cx];
    const ray = [ux, uy, -FOCAL];
    const denom = normal[0] * ray[0] + normal[1] * ray[1] + normal[2] * ray[2];
    const t = -(normal[2] * FOCAL) / (Math.abs(denom) > 0.00001 ? denom : -0.00001);
    if (t <= 0) return [99, 99];
    const q = [ray[0] * t, ray[1] * t, FOCAL + ray[2] * t];
    ux = q[0] * cy - q[2] * sy;
    uy = q[0] * sy * sx + q[1] * cx + q[2] * cy * sx;
  }
  const sin = Math.sin(state.rotation);
  const cos = Math.cos(state.rotation);
  return [cos * ux - sin * uy, sin * ux + cos * uy];
};

const render = () => {
  const s = settingsRef.current;
  const [c1, c2, c3] = s.colors.map(value => shiftHue(parse(value), s.hue));
  const reveal = state.reveal;
  const back = reveal - 1;
  const grow = 1 + 2.2 * back * back * back + 1.2 * back * back;
  gl.viewport(0, 0, canvas.width, canvas.height);
  gl.useProgram(program.program);
  gl.bindVertexArray(vao);
  gl.uniform2f(u.uResolution, canvas.width, canvas.height);
  gl.uniform1f(u.uTime, state.time);
  gl.uniform2f(u.uReach, state.reachX, state.reachY);
  gl.uniform1f(u.uNear, state.near);
  gl.uniform1f(u.uAwake, state.hover * s.hoverIntensity * 0.18);
  gl.uniform1f(u.uRotation, state.rotation);
  gl.uniform3fv(u.uBackground, parse(s.backgroundColor));
  gl.uniform3fv(u.uColor1, c1);
  gl.uniform3fv(u.uColor2, c2);
  gl.uniform3fv(u.uColor3, c3);
  gl.uniform1f(u.uScale, 1 / (s.size * (1 + state.pulse) * (0.62 + 0.38 * grow)));
  gl.uniform1f(u.uWobble, s.wobble);
  gl.uniform1f(u.uGlow, s.glow);
  gl.uniform1f(u.uBleed, s.bleed);
  gl.uniform1f(u.uHighlight, s.highlight);
  gl.uniform2f(u.uEcho, state.echo * 0.55, state.echo < 1 ? (1 - state.echo) * (1 - state.echo) : 0);
  gl.uniform1f(u.uSwirl, state.swirl);
  gl.uniform1f(u.uOrbit, state.orbit);
  gl.uniform1f(
    u.uReveal,
    Math.min(1, reveal / 0.55) * Math.min(1, reveal / 0.55) * (3 - 2 * Math.min(1, reveal / 0.55))
  );
  gl.uniform1f(u.uOpacity, s.opacity);
```

```
  gl.uniform1f(u.uTrail, s.trail);
  gl.uniform2f(u.uTilt, (-s.tiltX * Math.PI) / 180, (s.tiltY * Math.PI) / 180);
  gl.drawArrays(gl.TRIANGLES, 0, 3);
 };

 const frame = now => {
  raf = 0;
  if (!visible || document.hidden) return;
  const s = settingsRef.current;
  const dt = last ? clamp((now - last) / 1000, 0.0005, 0.05) : 1 / 60;
  last = now;
  if (s.paused) {
   state.reveal = 1;
   render();
   last = 0;
   return;
  }
  const motion = reduce ? 0 : 1;
  const [px, py] = toOrb(pointer.x, pointer.y);
  const distance = Math.hypot(px, py);
  const region = pointer.inside && motion ? clamp((1.45 - distance) / 0.4, 0, 1) : 0;
  const zone = region * region * (3 - 2 * region);
  const aimed = zone * clamp((distance - 0.15) / 0.45, 0, 1);
  const near = zone > 0.5;
  const target = s.forceHoverState ? 1 : zone;
  state.hover += (target - state.hover) * (1 - Math.exp(-dt / 0.16));
  const reach = 0.075 * s.hoverIntensity * aimed;
  const goalX = distance > 0.0001 ? (px / distance) * reach : 0;
  const goalY = distance > 0.0001 ? (py / distance) * reach : 0;
  state.reachSpeedX += (70 * (goalX - state.reachX) - 7 * state.reachSpeedX) * dt;
  state.reachSpeedY += (70 * (goalY - state.reachY) - 7 * state.reachSpeedY) * dt;
  state.reachX += state.reachSpeedX * dt;
  state.reachY += state.reachSpeedY * dt;
  state.near += (aimed * s.hoverIntensity - state.near) * (1 - Math.exp(-dt / 0.12));

  const step = dt * s.speed * motion;
  state.time += step;
  state.swirl += step * 2 * (1 + state.kick);
  state.echo = Math.min(1, state.echo + dt / 1.2);
  if (s.rotateOnHover) state.rotation += dt * 0.3 * state.hover * motion;

  const aim = Math.atan2(py, px);
  if (s.followCursor && near && motion) {
   const delta = Math.atan2(Math.sin(aim - state.orbit), Math.cos(aim - state.orbit));
   state.orbit += delta * (1 - Math.exp(-dt * 6));
  } else {
   state.orbit -= step;
  }
  state.orbit = Math.atan2(Math.sin(state.orbit), Math.cos(state.orbit));

  state.pulseSpeed += (-170 * state.pulse - 11 * state.pulseSpeed) * dt;
  state.pulse += state.pulseSpeed * dt;
  state.kick *= Math.exp(-dt * 2.5);

  if (!s.intro || reduce) state.reveal = 1;
  else if (state.reveal < 1) state.reveal = Math.min(1, state.reveal + dt / INTRO_SECONDS);

  render();
  raf = requestAnimationFrame(frame);
 };

 const wake = () => {
  if (!raf && visible && !document.hidden) raf = requestAnimationFrame(frame);
 };

 const resize = () => {
  const s = settingsRef.current;
  const width = Math.max(1, container.clientWidth);
  const height = Math.max(1, container.clientHeight);
  const ratio = Math.min(s.dpr || window.devicePixelRatio || 1, 2);
  view.width = width;
  view.height = height;
  canvas.width = Math.max(1, Math.round(width * ratio));
  canvas.height = Math.max(1, Math.round(height * ratio));
```

```
  render();
  wake();
};

const locate = event => {
  const rect = container.getBoundingClientRect();
  pointer.x = event.clientX - rect.left;
  pointer.y = event.clientY - rect.top;
  pointer.inside = pointer.x >= 0 && pointer.y >= 0 && pointer.x <= rect.width && pointer.y <= rect.height;
};

const onPointerMove = event => {
  locate(event);
  if (!settingsRef.current.paused) wake();
};

const onPointerDown = event => {
  const s = settingsRef.current;
  locate(event);
  if (!pointer.inside || !s.clickPulse || s.paused || reduce) return;
  const [px, py] = toOrb(pointer.x, pointer.y);
  if (Math.hypot(px, py) > 1) return;
  state.pulseSpeed += 0.9;
  state.kick = 2.2;
  state.echo = 0;
  wake();
};

const onPointerUp = event => {
  if (event.pointerType !== 'mouse') pointer.inside = false;
};

const onPointerLeave = () => {
  pointer.inside = false;
};

const onVisibility = () => {
  last = 0;
  wake();
};

const resizeObserver = new ResizeObserver(resize);
resizeObserver.observe(container);
const intersection = new IntersectionObserver(entries => {
  visible = entries.some(entry => entry.isIntersecting);
  if (visible) {
    last = 0;
    wake();
  }
});
intersection.observe(container);
window.addEventListener('pointermove', onPointerMove, { passive: true });
window.addEventListener('pointerdown', onPointerDown, { passive: true });
window.addEventListener('pointerup', onPointerUp, { passive: true });
window.addEventListener('pointercancel', onPointerUp, { passive: true });
document.documentElement.addEventListener('pointerleave', onPointerLeave);
document.addEventListener('visibilitychange', onVisibility);
resize();

let lastDpr = settingsRef.current.dpr;
wakeRef.current = () => {
  const s = settingsRef.current;
  if (s.dpr !== lastDpr) {
    lastDpr = s.dpr;
    resize();
    return;
  }
  if (!raf) render();
  wake();
};

return () => {
  cancelAnimationFrame(raf);
  wakeRef.current = null;
```

```
      resizeObserver.disconnect();
      intersection.disconnect();
      window.removeEventListener('pointermove', onPointerMove);
      window.removeEventListener('pointerdown', onPointerDown);
      window.removeEventListener('pointerup', onPointerUp);
      window.removeEventListener('pointercancel', onPointerUp);
      document.documentElement.removeEventListener('pointerleave', onPointerLeave);
      document.removeEventListener('visibilitychange', onVisibility);
      gl.deleteProgram(program.program);
      gl.deleteBuffer(buffer);
      gl.deleteVertexArray(vao);
      gl.getExtension('WEBGL_lose_context')?.loseContext();
    };
  }, []);

  useEffect(() => {
    wakeRef.current?.();
  });

  return (
    <div ref={containerRef} className={`orb-container${className ? ` ${className}` : ''}`} style={style}>
      <canvas ref={canvasRef} className="orb-canvas" aria-hidden="true" />
    </div>
  );
};

export default Orb;

```

Component CSS
```css
.orb-container {
  position: relative;
  z-index: 0;
  width: 100%;
  height: 100%;
  overflow: hidden;
}

.orb-canvas {
  position: absolute;
  inset: 0;
  display: block;
  width: 100%;
  height: 100%;
}

```

Integration Instructions
1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

More from React Bits
The full library index, including everything reactbits.dev offers, is at https://reactbits.dev/llms.txt — fetch it if this component is not the right fit or the project needs more pieces.

Header Buttons and animation-

Integrate the <GooeyNav /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: GooeyNav
Variant: JavaScript + CSS

Usage Example
```jsx
import GooeyNav from './GooeyNav';

const items = [
  { label: 'Home', href: '#' },
  { label: 'Work', href: '#' },
  { label: 'About', href: '#' },
  { label: 'Contact', href: '#' }
];

<GooeyNav
  items={items}
  initialActiveIndex={0}
  theme="dark"
  size="md"
  frame={true}
  particleCount={15}
  spread={56}
  animationTime={600}
  timeVariance={300}
  gooeyness={0.5}
  wobble={0.5}
  hoverEffect={true}
  onChange={(index, item) => console.log(index, item.label)}
/>
```

Props
Prop	Type	Default	Description
items	{ label: string; href?: string; icon?: ReactNode }[]	-	The navigation items. Items without an href render as buttons.
initialActiveIndex	number	0	Which item starts selected.
activeIndex	number	-	Controls the selected item yourself. Use it together with onChange.
onChange	(index: number, item) => void	-	Called when an item is selected.
theme	'dark' | 'light' | 'dark'	Color theme of the frame and text.
color	string	-	Color of the active pill. Uses a high-contrast color for the theme when not set.
activeTextColor	string	-	Text color inside the pill. Picked automatically from the pill color when not set.
colors	string[]	[]	Colors for the droplets. They blend as they merge. Uses the pill color when empty.
size	'sm' | 'md' | 'lg'	'md'	Size of the items.
frame	boolean	true	Shows the frosted panel around the items.
animationTime	number	600	Roughly how long the droplets take to travel to the new item, in ms.
particleCount	number	15	How many droplets the pill breaks into when an item is selected.
spread	number	56	How far the droplets scatter on their way to the new item, in px.
timeVariance	number	300	Random variation in each droplet's timing, in ms.
gooeyness	number	0.5	How readily the droplets and pills melt together, from 0 to 1.
wobble	number	0.5	How much the new pill wobbles as it forms, from 0 to 1.
hoverEffect	boolean	true	Shows a soft highlight behind the hovered item.
className	string	''	Extra class names for the nav.
style	CSSProperties	-	Inline styles for the nav.

Full Component Source
```jsx
'use client';

import { useEffect, useLayoutEffect, useRef, useState } from 'react';

import './GooeyNav.css';

const useIsomorphicLayoutEffect = typeof window === 'undefined' ? useEffect : useLayoutEffect;

const THEMES = {
  dark: {
    '--gn-frame': 'rgba(38, 35, 46, 0.66)',
    '--gn-frame-edge': 'rgba(255, 255, 255, 0.08)',
    '--gn-frame-highlight': 'rgba(255, 255, 255, 0.07)',
    '--gn-frame-shadow': '0 22px 44px -20px rgba(0, 0, 0, 0.8), 0 6px 16px -8px rgba(0, 0, 0, 0.5)',
    '--gn-ink': '#f4f4f5',
    '--gn-muted': 'rgba(244, 244, 245, 0.62)'
  },
  light: {
```

```
    '--gn-frame': 'rgba(240, 240, 243, 0.8)',
    '--gn-frame-edge': 'rgba(24, 24, 27, 0.07)',
    '--gn-frame-highlight': 'rgba(255, 255, 255, 0.95)',
    '--gn-frame-shadow': '0 22px 44px -22px rgba(24, 24, 27, 0.3), 0 6px 16px -10px rgba(24, 24, 27, 0.16)',
    '--gn-ink': '#27272a',
    '--gn-muted': 'rgba(39, 39, 42, 0.62)'
  }
};

const PILLS = {
  dark: { color: '#f4f4f5', hover: 'rgba(255, 255, 255, 0.08)', shadow: 0.4 },
  light: { color: '#27272a', hover: 'rgba(24, 24, 27, 0.06)', shadow: 0.2 }
};

const SIZES = { sm: 'gooey-nav--sm', md: 'gooey-nav--md', lg: 'gooey-nav--lg' };

const DEFAULT_ITEMS = [
  { label: 'Home', href: '#' },
  { label: 'Work', href: '#' },
  { label: 'About', href: '#' },
  { label: 'Contact', href: '#' }
];

const MAX_DROPS = 32;
const MAX_PILLS = 4;
const MARGIN = 72;

const VERTEX = `#version 300 es
in vec2 aPosition;
void main() {
  gl_Position = vec4(aPosition, 0.0, 1.0);
}
`;

const FRAGMENT = `#version 300 es
precision highp float;

uniform vec2 uSize;
uniform float uRatio;
uniform vec4 uPills[${MAX_PILLS}];
uniform vec4 uHover;
uniform vec4 uDrops[${MAX_DROPS}];
uniform vec2 uTails[${MAX_DROPS}];
uniform vec3 uTints[${MAX_DROPS}];
uniform int uCount;
uniform float uSmooth;
uniform vec3 uPill;
uniform vec4 uHoverColor;
uniform float uShadow;

out vec4 outColor;

float sdPill(vec2 p, vec4 box) {
  vec2 extent = max(box.zw, vec2(0.0));
  float r = min(extent.x, extent.y);
  vec2 q = abs(p - box.xy) - extent + r;
  return length(max(q, 0.0)) + min(max(q.x, q.y), 0.0) - r;
}

float sdCapsule(vec2 p, vec2 a, vec2 b, float r) {
  vec2 pa = p - a;
  vec2 ba = b - a;
  float h = clamp(dot(pa, ba) / max(dot(ba, ba), 0.0001), 0.0, 1.0);
  return length(pa - ba * h) - r;
}

vec4 blend(vec4 a, vec4 b, float k) {
  float h = clamp(0.5 + 0.5 * (b.w - a.w) / k, 0.0, 1.0);
  return vec4(mix(b.rgb, a.rgb, h), mix(b.w, a.w, h) - k * h * (1.0 - h));
}

vec4 field(vec2 p) {
  vec4 shape = vec4(uPill, 1e5);
  for (int i = 0; i < ${MAX_PILLS}; i++) {
```

```
    vec4 pill = uPills[i];
    if (pill.z <= 0.0) continue;
    shape = blend(shape, vec4(uPill, sdPill(p, pill)), uSmooth);
  }
  for (int i = 0; i < ${MAX_DROPS}; i++) {
    if (i >= uCount) break;
    vec4 drop = uDrops[i];
    if (drop.z <= 0.0) continue;
    float d = sdCapsule(p, drop.xy, drop.xy - uTails[i], drop.z);
    if (d > shape.w + uSmooth) continue;
    shape = blend(shape, vec4(uTints[i], d), uSmooth);
  }
  return shape;
}

void main() {
  vec2 p = vec2(gl_FragCoord.x, uSize.y * uRatio - gl_FragCoord.y) / uRatio;
  vec4 shape = field(p);
  float d = shape.w;
  float aa = max(fwidth(d), 0.35);
  float body = clamp(0.5 - d / aa, 0.0, 1.0);

  vec2 grad = vec2(dFdx(d), -dFdy(d));
  vec2 n = length(grad) > 0.0001 ? normalize(grad) : vec2(0.0, 1.0);
  float rim = (1.0 - smoothstep(0.0, 1.6, -d)) * max(0.0, -n.y) * 0.3;
  vec3 fill = mix(shape.rgb, vec3(1.0), rim) * (1.0 - 0.06 * max(0.0, n.y) * (1.0 - smoothstep(0.0, 6.0, -d)));

  float lifted = field(p - vec2(0.0, 3.0)).w;
  float shadow = (1.0 - smoothstep(-3.0, 9.0, lifted)) * uShadow * (1.0 - body);

  float hover = clamp(0.5 - sdPill(p, uHover) / aa, 0.0, 1.0) * uHoverColor.a;

  vec4 color = vec4(0.0, 0.0, 0.0, shadow);
  color = vec4(uHoverColor.rgb * hover, hover) + color * (1.0 - hover);
  color = vec4(fill * body, body) + color * (1.0 - body);
  outColor = color;
}
`;

const parse = value => {
  const match = /^#?([0-9a-f]{3}|[0-9a-f]{6})$/i.exec(String(value).trim());
  if (match) {
    const hex = match[1].length === 3 ? match[1].replace(/./g, digit => digit + digit) : match[1];
    return [0, 2, 4].map(index => parseInt(hex.slice(index, index + 2), 16) / 255).concat(1);
  }
  const rgba = /rgba?(\([^)]+\))/i.exec(String(value));
  if (rgba) {
    const parts = rgba[1].split(',').map(part => parseFloat(part));
    return [parts[0] / 255, parts[1] / 255, parts[2] / 255, parts[3] ?? 1];
  }
  return [1, 1, 1, 1];
};

const contrastInk = rgb => (0.2126 * rgb[0] + 0.7152 * rgb[1] + 0.0722 * rgb[2] > 0.55 ? '#18161d' : '#ffffff');

const clamp01 = value => Math.min(1, Math.max(0, value));

const travelEase = t => {
  const x = clamp01(t);
  return x < 0.5 ? 4 * x * x * x : 1 - (-2 * x + 2) ** 3 / 2;
};

const bezier = (a, b, c, d, t) => {
  const u = 1 - t;
  return u * u * u * a + 3 * u * u * t * b + 3 * u * t * t * c + t * t * t * d;
};

const pillPath = (x, y, w, h) => {
  if (w <= 0.5 || h <= 0.5) return '';
  const r = Math.min(w, h) / 2;
  return `M${x + r},${y}H${x + w - r}A${r},${r} 0 0 1 ${x + w},${y + r}V${y + h - r}A${r},${r} 0 0 1 ${x + w - r},${y + h}H${x + r}A${r},${r} 0 0 1 ${x},${y + h - r}V${y + r}A${r},${r} 0 0 1 ${x + r},${y}Z`;
};

const compile = (gl, type, source) => {
```

```
  const shader = gl.createShader(type);
  gl.shaderSource(shader, source);
  gl.compileShader(shader);
  return shader;
};

const UNIFORMS = [
  'uSize',
  'uRatio',
  'uPills',
  'uHover',
  'uDrops',
  'uTails',
  'uTints',
  'uCount',
  'uSmooth',
  'uPill',
  'uHoverColor',
  'uShadow'
];

const build = gl => {
  const program = gl.createProgram();
  gl.attachShader(program, compile(gl, gl.VERTEX_SHADER, VERTEX));
  gl.attachShader(program, compile(gl, gl.FRAGMENT_SHADER, FRAGMENT));
  gl.linkProgram(program);
  if (!gl.getProgramParameter(program, gl.LINK_STATUS)) {
    gl.deleteProgram(program);
    return null;
  }
  const uniforms = Object.fromEntries(UNIFORMS.map(name => [name, gl.getUniformLocation(program, name)]));
  const buffer = gl.createBuffer();
  gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([-1, -1, 1, -1, -1, 1, 1, 1]), gl.STATIC_DRAW);
  return { program, uniforms, buffer, position: gl.getAttribLocation(program, 'aPosition') };
};

const GooeyNav = ({
  items = DEFAULT_ITEMS,
  initialActiveIndex = 0,
  activeIndex,
  onChange,
  theme = 'dark',
  color,
  activeTextColor,
  colors = [],
  size = 'md',
  frame = true,
  animationTime = 600,
  particleCount = 15,
  timeVariance = 300,
  spread = 56,
  gooeyness = 0.5,
  wobble = 0.5,
  hoverEffect = true,
  className = '',
  style
}) => {
  const [innerIndex, setInnerIndex] = useState(initialActiveIndex);
  const current = Math.min(Math.max(0, activeIndex ?? innerIndex), Math.max(0, items.length - 1));
  const stageRef = useRef(null);
  const canvasRef = useRef(null);
  const listRef = useRef(null);
  const overlayRef = useRef(null);
  const fallbackRef = useRef(null);
  const engineRef = useRef(null);
  const indexRef = useRef(current);
  indexRef.current = current;
  const palette = theme === 'light' ? 'light' : 'dark';
  const pillColor = color || PILLS[palette].color;
  const pillRgb = parse(pillColor);
  const settings = {
    animationTime,
    particleCount,
```

```
  timeVariance,
  spread,
  gooeyness,
  wobble,
  hoverEffect,
  pillRgb,
  tints: colors.map(parse),
  hoverRgb: parse(PILLS[palette].hover),
  shadow: PILLS[palette].shadow
};
const settingsRef = useRef(settings);
settingsRef.current = settings;

const select = index => {
  if (index === current) return;
  if (activeIndex === undefined) setInnerIndex(index);
  onChange?.(index, items[index]);
};

useIsomorphicLayoutEffect(() => {
  const stage = stageRef.current;
  const canvas = canvasRef.current;
  const list = listRef.current;
  const overlay = overlayRef.current;
  const fallback = fallbackRef.current;
  if (!stage || !canvas || !list || !overlay || !fallback) return;

  const reduced = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
  const gl = canvas.getContext('webgl2', { alpha: true, premultipliedAlpha: true, antialias: false });
  const pipeline = gl ? build(gl) : null;
  fallback.style.display = pipeline ? 'none' : 'block';
  canvas.style.display = pipeline ? 'block' : 'none';

  const view = { width: 0, height: 0 };
  let rects = [];
  let target = -1;
  const pills = new Map();
  const drops = [];
  const hover = { x: 0, y: 0, w: 0, h: 0, alpha: 0, index: -1 };
  const pillData = new Float32Array(MAX_PILLS * 4);
  const dropData = new Float32Array(MAX_DROPS * 4);
  const tailData = new Float32Array(MAX_DROPS * 2);
  const tintData = new Float32Array(MAX_DROPS * 3);
  let raf = 0;
  let last = 0;
  let disposed = false;

  const measure = () => {
    const box = stage.getBoundingClientRect();
    const ratio = Math.min(window.devicePixelRatio || 1, 2);
    view.width = box.width;
    view.height = box.height;
    const outerWidth = box.width + MARGIN * 2;
    const outerHeight = box.height + MARGIN * 2;
    Object.assign(canvas.style, {
      left: `${-MARGIN}px`,
      top: `${-MARGIN}px`,
      width: `${outerWidth}px`,
      height: `${outerHeight}px`
    });
    const width = Math.max(1, Math.round(outerWidth * ratio));
    const height = Math.max(1, Math.round(outerHeight * ratio));
    if (canvas.width !== width || canvas.height !== height) {
      canvas.width = width;
      canvas.height = height;
    }
    rects = Array.from(list.children).map(item => {
      const rect = item.getBoundingClientRect();
      return { x: rect.left - box.left, y: rect.top - box.top, w: rect.width, h: rect.height };
    });
  };

  const trace = (boxes, blobs, smooth) => {
    const field = (x, y) => {
```

```
    let d = 1e5;
    const merge = value => {
      const h = clamp01(0.5 + (0.5 * (value - d)) / smooth);
      d = value + (d - value) * h - smooth * h * (1 - h);
    };
    for (const box of boxes) {
      const r = Math.min(box.hw, box.hh);
      const qx = Math.abs(x - box.cx) - box.hw + r;
      const qy = Math.abs(y - box.cy) - box.hh + r;
      merge(Math.hypot(Math.max(qx, 0), Math.max(qy, 0)) + Math.min(Math.max(qx, qy), 0) - r);
    }
    for (const blob of blobs) {
      const px = x - blob.x;
      const py = y - blob.y;
      const bx = -blob.tx;
      const by = -blob.ty;
      const span = bx * bx + by * by;
      const h = span > 0.0001 ? clamp01((px * bx + py * by) / span) : 0;
      const value = Math.hypot(px - bx * h, py - by * h) - blob.r;
      if (value > d + smooth) continue;
      merge(value);
    }
    return d;
  };
  const step = 2;
  const parts = [];
  for (let y = 0; y < view.height; y += 1) {
    const cy = y + 0.5;
    let previous = field(0, cy);
    let start = previous < 0 ? 0 : null;
    for (let x = step; x <= view.width + step; x += step) {
      const value = field(Math.min(x, view.width), cy);
      if (previous < 0 !== value < 0) {
        const cross = x - step + (step * previous) / (previous - value);
        if (value < 0) start = cross;
        else if (start !== null) {
          parts.push(`M${start.toFixed(1)},${y}H${cross.toFixed(1)}V${y + 1}H${start.toFixed(1)}Z`);
          start = null;
        }
      }
      previous = value;
    }
    if (start !== null) parts.push(`M${start.toFixed(1)},${y}H${view.width.toFixed(1)}V${y + 1}H${start.toFixed(1)}Z`);
  }
  return parts.join('');
};

const pill = index => {
  if (!pills.has(index)) pills.set(index, { mass: 0, scale: 0, velocity: 0, outgoing: 0 });
  return pills.get(index);
};

const shape = (index, amount) => {
  const rect = rects[index];
  if (!rect || amount <= 0.002) return null;
  const h = rect.h * Math.min(1.08, 0.42 + 0.58 * Math.sqrt(Math.max(0, amount)));
  const w = Math.max(h, rect.w * Math.max(0, amount) ** 0.75);
  return { x: rect.x + rect.w / 2 - w / 2, y: rect.y + rect.h / 2 - h / 2, w, h };
};

const plan = (drop, index, now, fresh) => {
  const s = settingsRef.current;
  const rect = rects[index];
  if (!rect) return;
  const reach = Math.max(0, s.spread);
  const ex = rect.x + rect.w / 2 + (Math.random() - 0.5) * rect.w * 0.45;
  const ey = rect.y + rect.h / 2 + (Math.random() - 0.5) * rect.h * 0.2;
  const base = Math.max(0.15, s.animationTime / 1000);
  const variance = (Math.random() - 0.5) * (Math.max(0, s.timeVariance) / 1000);
  const duration = Math.max(0.15, base * (0.62 + Math.random() * 0.32) + variance * 0.6);
  const out = Math.random() * Math.PI * 2;
  const into = Math.random() * Math.PI * 2;
  const burst = reach * (0.7 + Math.random() * 0.6);
  const swing = reach * (0.55 + Math.random() * 0.55);
```

```
  const carry = Math.min(duration / 3, 0.12);
  drop.index = index;
  drop.start = now;
  drop.duration = duration;
  drop.path = [
    drop.x,
    drop.y,
    fresh ? drop.x + Math.cos(out) * burst * 1.6 : drop.x + drop.vx * carry,
    fresh ? drop.y + Math.sin(out) * burst * 1.6 : drop.y + drop.vy * carry,
    ex + Math.cos(into) * swing * 1.5,
    ey + Math.sin(into) * swing * 1.5,
    ex,
    ey
  ];
};

const emit = (from, to, now) => {
  const s = settingsRef.current;
  const source = pills.get(from);
  const rect = rects[from];
  if (!source || !rect || !rects[to]) return;
  const available = source.mass - source.outgoing;
  if (available <= 0.002) return;
  const room = MAX_DROPS - drops.length;
  const count = Math.min(room, Math.max(1, Math.round(Math.max(1, s.particleCount) * available)));
  if (count <= 0) return;
  const window = Math.max(0.15, s.animationTime / 1000) * 0.16;
  for (let i = 0; i < count; i++) {
    const mass = available / count;
    source.outgoing += mass;
    drops.push({
      x: rect.x + rect.w / 2 + (Math.random() - 0.5) * rect.w * 0.6,
      y: rect.y + rect.h / 2 + (Math.random() - 0.5) * rect.h * 0.3,
      vx: 0,
      vy: 0,
      mass,
      size: 0.8 + Math.random() * 0.4,
      given: 0,
      source: from,
      index: to,
      leave: now + Math.random() * window * 1000,
      emerged: 0,
      absorbed: 0,
      state: 'waiting',
      tint: s.tints.length ? s.tints[Math.floor(Math.random() * s.tints.length)] : null
    });
  }
};

const render = () => {
  const s = settingsRef.current;
  const height = rects[target]?.h ?? rects[0]?.h ?? 36;
  const smooth = height * (0.18 + 0.5 * clamp01(s.gooeyness));
  const boxes = [];
  const blobs = [];
  for (const [index, state] of pills) {
    const box = shape(index, state.scale);
    if (box) boxes.push({ cx: box.x + box.w / 2, cy: box.y + box.h / 2, hw: box.w / 2, hh: box.h / 2 });
  }
  for (const drop of drops) {
    if (drop.state === 'waiting') continue;
    const radius = 0.26 * height * drop.size * Math.min(drop.emerged, 1 - drop.absorbed);
    if (radius <= 0.05) continue;
    const speed = Math.hypot(drop.vx, drop.vy);
    const length = Math.min(radius * 1.1, speed * 0.018);
    blobs.push({
      x: drop.x,
      y: drop.y,
      r: radius,
      tx: speed > 0.01 ? (drop.vx / speed) * length : 0,
      ty: speed > 0.01 ? (drop.vy / speed) * length : 0,
      tint: drop.tint
    });
  }
```

```
  const settledPill = boxes.length === 1 && !blobs.length;
  if (settledPill) {
    const box = boxes[0];
    overlay.style.clipPath = `path('${pillPath(box.cx - box.hw, box.cy - box.hh, box.hw * 2, box.hh * 2)}')`;
  } else {
    overlay.style.clipPath = `path('${trace(boxes, blobs, smooth) || 'M0,0Z'}')`;
  }
  overlay.style.visibility = 'visible';

  if (!pipeline || !gl) {
    const box = shape(target, 1);
    if (box) {
      fallback.style.transform = `translate(${box.x.toFixed(2)}px, ${box.y.toFixed(2)}px)`;
      fallback.style.width = `${box.w}px`;
      fallback.style.height = `${box.h}px`;
    }
    fallback.style.opacity = box ? '1' : '0';
    return;
  }

  let slot = 0;
  for (const box of boxes) {
    if (slot >= MAX_PILLS) break;
    pillData[slot * 4] = box.cx + MARGIN;
    pillData[slot * 4 + 1] = box.cy + MARGIN;
    pillData[slot * 4 + 2] = box.hw;
    pillData[slot * 4 + 3] = box.hh;
    slot += 1;
  }
  pillData.fill(0, slot * 4);
  let count = 0;
  for (const blob of blobs) {
    if (count >= MAX_DROPS) break;
    dropData[count * 4] = blob.x + MARGIN;
    dropData[count * 4 + 1] = blob.y + MARGIN;
    dropData[count * 4 + 2] = blob.r;
    dropData[count * 4 + 3] = 0;
    tailData[count * 2] = blob.tx;
    tailData[count * 2 + 1] = blob.ty;
    const tint = blob.tint || s.pillRgb;
    tintData[count * 3] = tint[0];
    tintData[count * 3 + 1] = tint[1];
    tintData[count * 3 + 2] = tint[2];
    count += 1;
  }

  const { program, uniforms, buffer, position } = pipeline;
  gl.viewport(0, 0, canvas.width, canvas.height);
  gl.clearColor(0, 0, 0, 0);
  gl.clear(gl.COLOR_BUFFER_BIT);
  gl.useProgram(program);
  gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
  gl.enableVertexAttribArray(position);
  gl.vertexAttribPointer(position, 2, gl.FLOAT, false, 0, 0);
  gl.uniform2f(uniforms.uSize, view.width + MARGIN * 2, view.height + MARGIN * 2);
  gl.uniform1f(uniforms.uRatio, canvas.width / Math.max(1, view.width + MARGIN * 2));
  gl.uniform4fv(uniforms.uPills, pillData);
  gl.uniform4f(uniforms.uHover, hover.x + MARGIN, hover.y + MARGIN, hover.w / 2, hover.h / 2);
  gl.uniform4fv(uniforms.uDrops, dropData);
  gl.uniform2fv(uniforms.uTails, tailData);
  gl.uniform3fv(uniforms.uTints, tintData);
  gl.uniform1i(uniforms.uCount, count);
  gl.uniform1f(uniforms.uSmooth, smooth);
  gl.uniform3f(uniforms.uPill, s.pillRgb[0], s.pillRgb[1], s.pillRgb[2]);
  gl.uniform4f(uniforms.uHoverColor, s.hoverRgb[0], s.hoverRgb[1], s.hoverRgb[2], s.hoverRgb[3] * hover.alpha);
  gl.uniform1f(uniforms.uShadow, s.shadow);
  gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
};

const frame = now => {
  raf = 0;
  const s = settingsRef.current;
  const dt = Math.min(0.05, (now - last) / 1000 || 0);
```

```
last = now;
const emergeTime = 0.09;
const absorbTime = 0.12;

for (let i = drops.length - 1; i >= 0; i--) {
  const drop = drops[i];
  if (drop.state === 'waiting') {
    if (now < drop.leave) continue;
    drop.state = 'flying';
    plan(drop, drop.index, now, true);
  }
  if (drop.state === 'flying') {
    drop.emerged = Math.min(1, drop.emerged + dt / emergeTime);
    const source = pills.get(drop.source);
    if (source && drop.given < drop.mass) {
      const step = Math.min(drop.mass * (dt / emergeTime), drop.mass - drop.given);
      source.mass = Math.max(0, source.mass - step);
      source.outgoing = Math.max(0, source.outgoing - step);
      drop.given += step;
    }
    const t = clamp01((now - drop.start) / 1000 / drop.duration);
    const eased = travelEase(t);
    const [x0, y0, x1, y1, x2, y2, x3, y3] = drop.path;
    const x = bezier(x0, x1, x2, x3, eased);
    const y = bezier(y0, y1, y2, y3, eased);
    if (dt > 0) {
      drop.vx = (x - drop.x) / dt;
      drop.vy = (y - drop.y) / dt;
    }
    drop.x = x;
    drop.y = y;
    if (t >= 1) drop.state = 'absorbing';
  }
  if (drop.state === 'absorbing') {
    const step = Math.min(1 - drop.absorbed, dt / absorbTime);
    drop.absorbed += step;
    drop.vx *= 0.8;
    drop.vy *= 0.8;
    const host = pill(drop.index);
    host.mass = Math.min(1, host.mass + drop.mass * step);
    if (drop.absorbed >= 1) drops.splice(i, 1);
  }
}

const zeta = reduced ? 1 : 1 - 0.72 * clamp01(s.wobble);
const omega = 26;
const steps = Math.max(1, Math.ceil(dt / (1 / 240)));
const h = dt / steps;
for (const [index, state] of pills) {
  for (let n = 0; n < steps; n++) {
    state.velocity += (omega * omega * (state.mass - state.scale) - 2 * zeta * omega * state.velocity) * h;
    state.scale += state.velocity * h;
  }
  if (index !== target && state.mass <= 0.002 && state.outgoing <= 0.002 && Math.abs(state.scale) < 0.004) {
    pills.delete(index);
  }
}

if (hover.index >= 0 && rects[hover.index]) {
  const rect = rects[hover.index];
  const k = 1 - Math.exp(-dt / 0.06);
  hover.x += (rect.x + rect.w / 2 - hover.x) * k;
  hover.y += (rect.y + rect.h / 2 - hover.y) * k;
  hover.w += (rect.w - hover.w) * k;
  hover.h += (rect.h - hover.h) * k;
}
const hoverGoal = s.hoverEffect && hover.index >= 0 && hover.index !== target ? 1 : 0;
hover.alpha += (hoverGoal - hover.alpha) * (1 - Math.exp(-dt / 0.08));

render();

const host = pills.get(target);
const resting =
  drops.length === 0 &&
```

```
    pills.size === 1 &&
    host &&
    Math.abs(host.mass - host.scale) < 0.0008 &&
    Math.abs(host.velocity) < 0.004;
  const hoverRect = rects[hover.index];
  const hovering =
    Math.abs(hoverGoal - hover.alpha) > 0.003 ||
    (hoverRect && Math.abs(hoverRect.x + hoverRect.w / 2 - hover.x) + Math.abs(hoverRect.w - hover.w) > 0.1);
  if (!resting || hovering) raf = requestAnimationFrame(frame);
  else {
    host.mass = 1;
    host.scale = 1;
    host.velocity = 0;
    hover.alpha = hoverGoal;
    render();
  }
};

const wake = () => {
  if (raf || disposed) return;
  last = performance.now();
  raf = requestAnimationFrame(frame);
};

const settle = index => {
  pills.clear();
  drops.length = 0;
  target = index;
  const state = pill(index);
  state.mass = 1;
  state.scale = 1;
  state.velocity = 0;
  render();
};

const go = (index, animate) => {
  if (index === target) return;
  if (!rects[index] || !animate || reduced || target < 0) {
    settle(index);
    return;
  }
  const now = performance.now();
  target = index;
  pill(index);
  for (const drop of drops) {
    drop.index = index;
    if (drop.state === 'flying') plan(drop, index, now, false);
    if (drop.state === 'absorbing') {
      drop.state = 'flying';
      drop.mass *= 1 - drop.absorbed;
      drop.absorbed = 0;
      plan(drop, index, now, false);
    }
  }
  for (const from of [...pills.keys()]) {
    if (from !== index) emit(from, index, now);
  }
  wake();
};

const onOver = event => {
  const item = event.target.closest('[data-gooey-index]');
  if (!item || !list.contains(item)) return;
  const index = Number(item.dataset.gooeyIndex);
  const rect = rects[index];
  if (!rect) return;
  if (hover.alpha < 0.01) {
    hover.x = rect.x + rect.w / 2;
    hover.y = rect.y + rect.h / 2;
    hover.w = rect.w;
    hover.h = rect.h;
  }
  hover.index = index;
  wake();
```

```
  };

  const onLeave = () => {
    hover.index = -1;
    wake();
  };

  const resizeObserver = new ResizeObserver(() => {
    measure();
    if (!drops.length) settle(target < 0 ? indexRef.current : target);
    else render();
  });
  resizeObserver.observe(stage);
  resizeObserver.observe(list);

  list.addEventListener('pointerover', onOver);
  list.addEventListener('pointerleave', onLeave);

  measure();
  settle(indexRef.current);
  document.fonts?.ready.then(() => {
    if (disposed) return;
    measure();
    if (!drops.length) settle(target);
  });

  engineRef.current = {
    go: index => go(index, true),
    refresh: () => {
      render();
      wake();
    }
  };

  return () => {
    disposed = true;
    cancelAnimationFrame(raf);
    resizeObserver.disconnect();
    list.removeEventListener('pointerover', onOver);
    list.removeEventListener('pointerleave', onLeave);
    if (gl && pipeline) {
      gl.deleteBuffer(pipeline.buffer);
      gl.deleteProgram(pipeline.program);
    }
    engineRef.current = null;
  };
}, [items.length]);

useEffect(() => {
  engineRef.current?.go(current);
}, [current]);

useEffect(() => {
  engineRef.current?.refresh();
}, [pillColor, palette, gooeyness, size, frame]);

const textColor = activeTextColor || contrastInk(pillRgb);

const labels = interactive =>
  items.map((item, index) => {
    const content = (
      <>
        {item.icon && <span className="gooey-nav__icon">{item.icon}</span>}
        {item.label && <span className="gooey-nav__label">{item.label}</span>}
      </>
    );
    if (!interactive) {
      return (
        <li key={index} className="gooey-nav__item">
          <span className="gooey-nav__link">{content}</span>
        </li>
      );
    }
    const shared = {
```

```
        className: 'gooey-nav__link',
        'aria-current': index === current ? 'page' : undefined,
        onClick: event => {
          if (!item.href || item.href === '#') event.preventDefault();
          select(index);
        }
      };
      return (
        <li
          key={index}
          className="gooey-nav__item"
          data-gooey-index={index}
          data-active={index === current ? '' : undefined}
        >
          {item.href ? (
            <a href={item.href} {...shared}>
              {content}
            </a>
          ) : (
            <button type="button" {...shared}>
              {content}
            </button>
          )}
        </li>
      );
    });

  return (
    <nav
      className={`gooey-nav ${SIZES[size] ?? SIZES.md} ${frame ? ' gooey-nav--framed' : ''} ${className}`.trim()}
      style={{ ...THEMES[palette], '--gn-pill': pillColor, '--gn-active-ink': textColor, ...style }}
    >
      {frame && <span className="gooey-nav__rim" />}
      <div ref={stageRef} className="gooey-nav__stage">
        <canvas ref={canvasRef} className="gooey-nav__canvas" aria-hidden="true" />
        <span ref={fallbackRef} className="gooey-nav__fallback" aria-hidden="true" />
        <ul ref={listRef} className="gooey-nav__list">
          {labels(true)}
        </ul>
        <ul ref={overlayRef} className="gooey-nav__list gooey-nav__list--active" aria-hidden="true">
          {labels(false)}
        </ul>
      </div>
    </nav>
  );
};

export default GooeyNav;

```

Component CSS
```css
.gooey-nav {
  --gn-pad: 6px;
  --gn-gap: 4px;
  --gn-font: 14px;
  --gn-x: 16px;
  --gn-y: 9px;
  position: relative;
  display: inline-flex;
  max-width: 100%;
  font-size: var(--gn-font);
  line-height: 1.2;
}

.gooey-nav--sm {
  --gn-pad: 5px;
  --gn-gap: 2px;
  --gn-font: 13px;
  --gn-x: 13px;
  --gn-y: 7px;
}
```

```
.gooey-nav--lg {
  --gn-pad: 7px;
  --gn-gap: 6px;
  --gn-font: 16px;
  --gn-x: 20px;
  --gn-y: 11px;
}

.gooey-nav--framed {
  padding: var(--gn-pad);
  border-radius: 999px;
  background: var(--gn-frame);
  box-shadow: var(--gn-frame-shadow);
  -webkit-backdrop-filter: blur(20px) saturate(1.6);
  backdrop-filter: blur(20px) saturate(1.6);
}

.gooey-nav__stage {
  position: relative;
  max-width: 100%;
}

.gooey-nav__canvas {
  position: absolute;
  top: 0;
  left: 0;
  pointer-events: none;
}

.gooey-nav__fallback {
  position: absolute;
  top: 0;
  left: 0;
  display: none;
  border-radius: 999px;
  background: var(--gn-pill);
  pointer-events: none;
}

.gooey-nav__list {
  position: relative;
  display: flex;
  flex-wrap: wrap;
  gap: var(--gn-gap);
  margin: 0;
  padding: 0;
  list-style: none;
}

.gooey-nav__list--active {
  position: absolute;
  inset: 0;
  color: var(--gn-active-ink);
  visibility: hidden;
  clip-path: inset(100%);
  pointer-events: none;
}

.gooey-nav__item {
  display: flex;
}

.gooey-nav__link {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: var(--gn-y) var(--gn-x);
  border: none;
  border-radius: 999px;
  background: none;
  color: var(--gn-muted);
  font: inherit;
  font-weight: 500;
  text-decoration: none;
```

```
  white-space: nowrap;
  cursor: pointer;
  transition: color 0.2s ease;
  -webkit-tap-highlight-color: transparent;
}

.gooey-nav__list--active .gooey-nav__link {
  color: inherit;
}

.gooey-nav__item[data-active] .gooey-nav__link {
  color: var(--gn-ink);
}

.gooey-nav__link:focus-visible {
  outline: 2px solid var(--gn-ink);
  outline-offset: 2px;
}

.gooey-nav__icon {
  display: inline-flex;
  flex: none;
}

.gooey-nav__icon svg {
  width: 1.15em;
  height: 1.15em;
}

.gooey-nav__rim {
  position: absolute;
  inset: 0;
  border-radius: inherit;
  box-shadow:
    inset 0 0 0 1px var(--gn-frame-edge),
    inset 0 1px 0 var(--gn-frame-highlight);
  pointer-events: none;
}

@media (hover: hover) {
  .gooey-nav__link:hover {
    color: var(--gn-ink);
  }
}
```

Integration Instructions
1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

More from React Bits
The full library index, including everything reactbits.dev offers, is at https://reactbits.dev/llms.txt — fetch it if this component is not the right fit or the project needs more pieces.

For sidebar menu-

Integrate the <undefined /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: undefined
Variant: JavaScript + CSS
Dependencies: @hugeicons/react @hugeicons/core-free-icons

Usage Example
```jsx
import { Download04Icon, Rocket01Icon, Settings02Icon } from '@hugeicons/core-free-icons';
import BranchedMenu from './BranchedMenu';

<BranchedMenu
  items={[
    {
      label: 'Getting started',
      children: [
        { value: 'install', label: 'Installation', icon: Download04Icon },
        { value: 'quick', label: 'Quick start', icon: Rocket01Icon },
        { value: 'config', label: 'Configuration', icon: Settings02Icon }
      ]
    },
    { label: 'Components', children: [{ value: 'buttons', label: 'Buttons' }, { value: 'overlays', label: 'Overlays' }] }
  ]}
  defaultOpen={[0]}
  defaultActive="quick"
  onSelect={(value, item) => navigate(value)}
  color="#f5f5f5"
  accentColor="#f5f5f5"
  lineColor="#3f3f46"
  width={240}
  rowHeight={36}
  indent={40}
  trunk={14}
  radius={10}
  lineWidth={1.5}
  fontSize={14}
  drawDuration={400}
  foldDuration={300}
/>
```

Props
Prop	Type	Default	Description
items	BranchedMenuItem[]	DEFAULT_ITEMS	Sections: label plus children (value, label, icon), or a leaf with a value. A section with children folds; a leaf selects.
defaultOpen	number	number[]	0	The section, or sections, open at first. -1 for none.
defaultActive	string	""	Value selected at first. Empty picks the open section's first child.
onSelect	(value, item) => void	-	A child or a leaf was picked.
onToggle	(index, open) => void	-	A section folded or unfolded.
color	string	"#f5f5f5"	The ink. Lines and idle text are mixes of it.
accentColor	string	"#f5f5f5"	The active line, the active label and the marker.
lineColor	string	"#3f3f46"	The rail, trunk and branches. A solid colour, so joints never darken.
width	number	240	The widest the menu may be, in px. It shrinks to its content.
rowHeight	number	36	Height of a child row in px.
indent	number	40	Where child rows start, in px. Branches end just before it.
trunk	number	14	The trunk line's x position, in px.
radius	number	10	The curve of each branch, in px.
lineWidth	number	1.5	Stroke width of the lines.
fontSize	number	14	Child text size in px. Headers are 1px larger.
drawDuration	number	400	The accent line's travel, in ms.
foldDuration	number	300	A section's fold and unfold, in ms.
className	string	""	Extra classes for the nav.

Full Component Source
```jsx
'use client';

import { isValidElement, useLayoutEffect, useRef, useState } from 'react';
import { HugeiconsIcon } from '@hugeicons/react';
import {
  CursorPointer01Icon,
  Download04Icon,
  Layers01Icon,
  Notification03Icon,
  PaintBoardIcon,
  Rocket01Icon,
  Settings02Icon,
  TextFontIcon
} from '@hugeicons/core-free-icons';
import './BranchedMenu.css';
```

```
const DEFAULT_ITEMS = [
  {
    label: 'Getting started',
    children: [
      { value: 'install', label: 'Installation', icon: Download04Icon },
      { value: 'quick', label: 'Quick start', icon: Rocket01Icon },
      { value: 'config', label: 'Configuration', icon: Settings02Icon },
      { value: 'theming', label: 'Theming', icon: PaintBoardIcon }
    ]
  },
  {
    label: 'Components',
    children: [
      { value: 'buttons', label: 'Buttons', icon: CursorPointer01Icon },
      { value: 'typography', label: 'Typography', icon: TextFontIcon },
      { value: 'overlays', label: 'Overlays', icon: Layers01Icon },
      { value: 'toasts', label: 'Toasts', icon: Notification03Icon }
    ]
  }
];
const PAD = 6;
const MARK = 16;

const renderIcon = icon => (isValidElement(icon) ? icon : <HugeiconsIcon icon={icon} size={16} strokeWidth={1.8} />);
const toSet = open => new Set(Array.isArray(open) ? open : open >= 0 ? [open] : []);

export default function BranchedMenu({
  items = DEFAULT_ITEMS,
  defaultOpen = 0,
  defaultActive = '',
  onSelect,
  onToggle,
  color = '#f5f5f5',
  accentColor = '#f5f5f5',
  lineColor = '#3f3f46',
  width = 240,
  rowHeight = 36,
  indent = 40,
  trunk = 14,
  radius = 10,
  lineWidth = 1.5,
  fontSize = 14,
  drawDuration = 400,
  foldDuration = 300,
  className = ''
}) {
  const [open, setOpen] = useState(() => toSet(defaultOpen));
  const [active, setActive] = useState(() => {
    if (defaultActive) return defaultActive;
    const first = items.find((it, i) => it.children && toSet(defaultOpen).has(i));
    return first?.children?.[0]?.value ?? '';
  });
  const navRef = useRef(null);
  const heads = useRef([]);
  const markerRef = useRef(null);
  const latest = useRef({});
  latest.current = { onSelect, onToggle };

  const activeSection = items.findIndex(it => it.children?.some(kid => kid.value === active));
  const markerShown = activeSection >= 0 && open.has(activeSection);
  useLayoutEffect(() => {
    const place = glide => {
      const m = markerRef.current;
      const el = heads.current[activeSection];
      if (!m) return;
      const on = markerShown && el;
      if (!glide) m.style.transition = 'none';
      if (on) m.style.top = `${el.offsetTop + (el.offsetHeight - MARK) / 2}px`;
      m.toggleAttribute('data-on', Boolean(on));
      if (!glide) {
        void m.offsetHeight;
        m.style.transition = '';
      }
```

```
  };
  place(true);
  let first = true;
  const ro = new ResizeObserver(() => {
    if (first) {
      first = false;
      return;
    }
    place(false);
  });
  if (navRef.current) ro.observe(navRef.current);
  return () => ro.disconnect();
}, [activeSection, markerShown, items, fontSize, rowHeight]);

const select = (value, item) => {
  setActive(value);
  latest.current.onSelect?.(value, item);
};
const toggle = i => {
  setOpen(prev => {
    const next = new Set(prev);
    const isOpen = !next.has(i);
    if (isOpen) next.add(i);
    else next.delete(i);
    latest.current.onToggle?.(i, isOpen);
    return next;
  });
};

const r = Math.min(radius, rowHeight / 2 - 2);
const endX = indent - 8;
const rowY = k => PAD + k * rowHeight + rowHeight / 2;
const branch = k => `M ${trunk} ${rowY(k) - r} A ${r} ${r} 0 0 0 ${trunk + r} ${rowY(k)} H ${endX}`;
const reach = k => `M ${trunk} 0 V ${rowY(k) - r} A ${r} ${r} 0 0 0 ${trunk + r} ${rowY(k)} H ${endX}`;
const length = k => rowY(k) - r + (Math.PI * r) / 2 + (endX - trunk - r);

return (
  <nav
    ref={navRef}
    className={`branched-menu${className ? ` ${className}` : ''}`}
    style={{
      '--bm-w': `${width}px`,
      '--bm-ink': color,
      '--bm-accent': accentColor,
      '--bm-line': lineColor,
      '--bm-font': `${fontSize}px`,
      '--bm-row': `${rowHeight}px`,
      '--bm-indent': `${indent}px`,
      '--bm-line-w': lineWidth,
      '--bm-draw': `${drawDuration}ms`,
      '--bm-fold': `${foldDuration}ms`
    }}
  >
    <span ref={markerRef} className="branched-menu__marker" aria-hidden="true" />
    {items.map((item, i) => {
      const kids = item.children;
      const isOpen = kids ? open.has(i) : false;
      const leafValue = item.value ?? item.label;
      const leafActive = !kids && leafValue === active;
      const bodyH = kids ? PAD * 2 + kids.length * rowHeight : 0;
      return (
        <div key={item.value ?? item.label} className="branched-menu__section" data-open={isOpen ? '' : undefined}>
          <button
            ref={el => {
              heads.current[i] = el;
            }}
            type="button"
            className="branched-menu__head"
            aria-expanded={kids ? isOpen : undefined}
            aria-current={leafActive ? 'true' : undefined}
            data-active={leafActive ? '' : undefined}
            onClick={() => (kids ? toggle(i) : select(leafValue, item))}
          >
            {item.label}
```

```
          </button>
          {kids ? (
            <div className="branched-menu__body">
              <div className="branched-menu__fold">
                <div className="branched-menu__tree" style={{ height: bodyH }}>
                  <svg className="branched-menu__lines" width={indent} height={bodyH} aria-hidden="true">
                    <path className="branched-menu__base" d={`M ${trunk} 0 V ${rowY(kids.length - 1) - r}`} />
                    {kids.map((kid, k) => (
                      <path key={kid.value} className="branched-menu__base" d={branch(k)} />
                    ))}
                    {kids.map((kid, k) => (
                      <path
                        key={kid.value}
                        className="branched-menu__reach"
                        d={reach(k)}
                        style={{
                          strokeDasharray: length(k),
                          strokeDashoffset: kid.value === active ? 0 : length(k)
                        }}
                      />
                    ))}
                  </svg>
                  {kids.map(kid => (
                    <button
                      key={kid.value}
                      type="button"
                      className="branched-menu__item"
                      aria-current={kid.value === active ? 'true' : undefined}
                      data-active={kid.value === active ? '' : undefined}
                      tabIndex={isOpen ? 0 : -1}
                      onClick={() => select(kid.value, kid)}
                    >
                      {kid.icon ? (
                        <span className="branched-menu__icon" aria-hidden="true">
                          {renderIcon(kid.icon)}
                        </span>
                      ) : null}
                      <span className="branched-menu__label">{kid.label}</span>
                    </button>
                  ))}
                </div>
              </div>
            </div>
          ) : null}
        </div>
      );
    })}
  </nav>
 );
}

```

Component CSS
```css
.branched-menu {
  --bm-w: 240px;
  --bm-ink: #f5f5f5;
  --bm-accent: #f5f5f5;
  --bm-line: #3f3f46;
  --bm-font: 14px;
  --bm-row: 36px;
  --bm-indent: 40px;
  --bm-line-w: 1.5;
  --bm-draw: 400ms;
  --bm-fold: 300ms;
  --bm-ease-out: cubic-bezier(0.23, 1, 0.32, 1);
  --bm-muted: color-mix(in srgb, var(--bm-ink) 55%, transparent);

  position: relative;
  display: flex;
  flex-direction: column;
  width: fit-content;
  max-width: min(var(--bm-w), 100%);
```

```
  padding-left: 14px;
  color: var(--bm-ink);
  font-family: inherit;
  font-size: var(--bm-font);
  line-height: 1.2;
}

.branched-menu::before {
  content: '';
  position: absolute;
  top: 8px;
  bottom: 0;
  left: 0;
  width: 2px;
  border-radius: 1px;
  background: linear-gradient(to bottom, var(--bm-line) 0%, var(--bm-line) 55%, transparent 100%);
}

.branched-menu__marker {
  position: absolute;
  top: -1px;
  left: 0;
  z-index: 1;
  width: 2px;
  height: 16px;
  border-radius: 1px;
  background: var(--bm-accent);
  opacity: 0;
  transition:
    top 220ms var(--bm-ease-out),
    opacity 150ms ease;
}

.branched-menu__marker[data-on] {
  opacity: 1;
}

.branched-menu__section {
  display: flex;
  flex-direction: column;
}

.branched-menu__head {
  display: block;
  margin: 0;
  padding: 9px 0;
  border: 0;
  background: none;
  color: var(--bm-muted);
  font: inherit;
  font-size: calc(var(--bm-font) + 1px);
  font-weight: 500;
  text-align: left;
  cursor: pointer;
  outline: none;
  -webkit-tap-highlight-color: transparent;
  transition: color 200ms ease;
}

.branched-menu__section[data-open] .branched-menu__head,
.branched-menu__head[data-active] {
  color: var(--bm-ink);
}

@media (hover: hover) and (pointer: fine) {
  .branched-menu__head:hover {
    color: var(--bm-ink);
  }
}

.branched-menu__body {
  display: grid;
  grid-template-rows: 0fr;
  transition: grid-template-rows var(--bm-fold) var(--bm-ease-out);
```

```
}

.branched-menu__section[data-open] .branched-menu__body {
  grid-template-rows: 1fr;
}

.branched-menu__fold {
  min-height: 0;
  overflow: hidden;
}

.branched-menu__tree {
  position: relative;
  padding: 6px 0;
  box-sizing: border-box;
}

.branched-menu__lines {
  position: absolute;
  top: 0;
  left: 0;
  overflow: visible;
  opacity: 0;
  pointer-events: none;
  transition: opacity 200ms ease;
}

.branched-menu__section[data-open] .branched-menu__lines {
  opacity: 1;
  transition: opacity 250ms ease 100ms;
}

.branched-menu__base,
.branched-menu__reach {
  fill: none;
  stroke-width: var(--bm-line-w);
  stroke-linecap: round;
  stroke-linejoin: round;
}

.branched-menu__base {
  stroke: var(--bm-line);
}

.branched-menu__reach {
  stroke: var(--bm-accent);
  transition: stroke-dashoffset var(--bm-draw) var(--bm-ease-out);
}

.branched-menu__item {
  display: flex;
  align-items: center;
  gap: 8px;
  width: 100%;
  height: var(--bm-row);
  margin: 0;
  padding: 0 0 0 var(--bm-indent);
  border: 0;
  background: none;
  color: var(--bm-muted);
  font: inherit;
  text-align: left;
  cursor: pointer;
  outline: none;
  box-sizing: border-box;
  -webkit-tap-highlight-color: transparent;
  transition: color 200ms ease;
}

@media (hover: hover) and (pointer: fine) {
  .branched-menu__item:hover {
    color: var(--bm-ink);
  }
}
```

```
.branched-menu__item[data-active] {
  color: var(--bm-accent);
  font-weight: 500;
}

.branched-menu__icon {
  display: inline-flex;
  flex: none;
}

.branched-menu__label {
  white-space: nowrap;
}

@media (prefers-reduced-motion: reduce) {
  .branched-menu__marker {
    transition: opacity 150ms ease;
  }

  .branched-menu__body {
    transition: none;
  }

  .branched-menu__reach {
    transition: none;
  }
}
```

Integration Instructions
1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

More from React Bits
The full library index, including everything reactbits.dev offers, is at https://reactbits.dev/llms.txt — fetch it if this component is not the right fit or the project needs more pieces.

Some other option buttons if needed-

Integrate the <JellyRadio /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: JellyRadio
Variant: JavaScript + CSS
Dependencies: motion

Usage Example
```jsx
import JellyRadio from './JellyRadio';

<JellyRadio
  items={['Off', 'Low', 'Medium', 'High', 'Max']}
  defaultValue="Medium"
  onChange={(value, index) => console.log(value, index)}
  chipColor="#27272a"
  activeColor="#f5f5f5"
  textColor="#f5f5f5"
  activeTextColor="#18181b"
  size="md"
  gap={8}
  radius={18}
  swell={0.2}
```

```
  barge={6}
  shrink={0.05}
  jelly={1}
  bounce={0.25}
  stagger={22}
  stiffness={580}
/>

<JellyRadio
  items={[
    { value: 'list', label: 'List', icon: <ListIcon /> },
    { value: 'grid', label: 'Grid', icon: <GridIcon /> },
    { value: 'map', label: 'Map', disabled: true }
  ]}
  defaultValue="grid"
/>
```

Props
Prop	Type	Default	Description
items	(string | { value: string; label: ReactNode; icon?: ReactNode; disabled?: boolean })[]	["Off", "Low", "Medium", "High", "Max"]	The chips. A string is both value and label.
value	string	undefined	Controlled value. Outside changes jump.
defaultValue	string	undefined	Initial value when uncontrolled. Falls back to the first item.
onChange	(value: string, index: number) => void	-	Called on the commit, before the motion ends.
chipColor	string	"#27272a"	Surface of an unchosen chip.
activeColor	string	"#f5f5f5"	Surface of the chosen chip.
textColor	string	"#f5f5f5"	Label of an unchosen chip, and the hover tone.
activeTextColor	string	"#18181b"	Label of the chosen chip.
size	"sm" | "md" | "lg"	"md"	Chip height 28, 36 or 44 pixels. Large is the touch-first size.
gap	number	8	Rest spacing between chips in pixels.
radius	number	18	Corner radius in pixels. 18 is a pill at medium.
swell	number	0.2	How much the chosen chip grows. It also sets the room the neighbours make.
barge	number	6	Extra pixels every neighbour is shoved beyond that room. 0 only makes room.
shrink	number	0.05	How much every unchosen chip gives up.
jelly	number	1	The wide-before-tall split. 0 swells uniformly, 1.5 exaggerates it.
bounce	number	0.25	One minus the damping ratio. 0 arrives and stops, 0.4 rings.
stagger	number	22	Milliseconds per row step before a neighbour moves. 0 moves the row as a slab.
stiffness	number	580	Spring stiffness of the chosen chip. Neighbours soften from it with distance.
disabled	boolean	false	Fades the group and ignores input.
ariaLabel	string	"Options"	Accessible name of the group.
className	string	""	Extra classes for the group.

Full Component Source
```jsx
'use client';

import { forwardRef, useEffect, useLayoutEffect, useRef, useState } from 'react';
import { animate, motion, motionValue, useReducedMotion, useTransform } from 'motion/react';

import './JellyRadio.css';

const DEFAULT_ITEMS = ['Off', 'Low', 'Medium', 'High', 'Max'];
const SIZES = { sm: [28, 12, 12], md: [36, 13, 16], lg: [44, 14, 20] };

const spring = (k, m, bounce) => ({
  type: 'spring',
  stiffness: k,
  damping: 2 * Math.sqrt(k * m) * (1 - bounce),
  mass: m
});

const Chip = forwardRef(function Chip({ mv, children, ...rest }, ref) {
  const transform = useTransform(() => `translateX(${mv.x.get()}px) scale(${mv.sx.get()}, ${mv.sy.get()})`);
  return (
    <motion.button ref={ref} style={{ transform }} {...rest}>
      {children}
    </motion.button>
  );
});

export default function JellyRadio({
  items = DEFAULT_ITEMS,
```

```
  value,
  defaultValue,
  onChange,
  chipColor = '#27272a',
  activeColor = '#f5f5f5',
  textColor = '#f5f5f5',
  activeTextColor = '#18181b',
  size = 'md',
  gap = 8,
  radius = 18,
  swell = 0.2,
  barge = 6,
  shrink = 0.05,
  jelly = 1,
  bounce = 0.25,
  stagger = 22,
  stiffness = 580,
  disabled = false,
  ariaLabel = 'Options',
  className = ''
}) {
  const list = items.map(it => (typeof it === 'string' ? { value: it, label: it } : it));
  const [inner, setInner] = useState(() => defaultValue ?? list[0]?.value);
  const current = value ?? inner;
  const at = Math.max(
    0,
    list.findIndex(it => it.value === current)
  );
  const reduce = useReducedMotion();
  const groupRef = useRef(null);
  const chipRefs = useRef([]);
  const widths = useRef([]);
  const mvs = useRef([]);
  const applied = useRef(at);
  const cfg = useRef({});
  cfg.current = { swell, barge, shrink, jelly, bounce, stagger, stiffness, reduce, count: list.length };
  const [h, font, px] = SIZES[size] ?? SIZES.md;
  const itemsKey = list.map(it => it.value).join('|');

  const mvFor = i => {
    let mv = mvs.current[i];
    if (!mv) {
      mv = { x: motionValue(0), sx: motionValue(1), sy: motionValue(1) };
      mvs.current[i] = mv;
    }
    return mv;
  };

  const apply = (sel, instant) => {
    const C = cfg.current;
    const group = groupRef.current;
    const rtl = group ? getComputedStyle(group).direction === 'rtl' : false;
    const push = ((widths.current[sel] ?? 0) * C.swell) / 2 + C.barge;
    for (let i = 0; i < C.count; i++) {
      const mv = mvFor(i);
      const on = i === sel;
      const far = Math.abs(i - sel);
      const dir = Math.sign(i - sel) * (rtl ? -1 : 1);
      const x = dir * push;
      const s = on ? 1 + C.swell : 1 - C.shrink;
      if (instant || C.reduce) {
        mv.x.jump(x);
        mv.sx.jump(s);
        mv.sy.jump(s);
        continue;
      }
      const k = C.stiffness * (1 - 0.12 * Math.min(far, 3));
      const inFlight = mv.x.isAnimating() || mv.sx.isAnimating() || mv.sy.isAnimating();
      const delay = inFlight ? 0 : (far * C.stagger) / 1000;
      animate(mv.x, x, { ...spring(k, 0.9, C.bounce), delay });
      const j = C.jelly;
      animate(mv.sx, s, {
        ...spring(k * (1 + 0.24 * j), 0.9 - 0.1 * j, Math.min(0.85, C.bounce + 0.3 * j)),
        delay
```

```
    });
    animate(mv.sy, s, { ...spring(k * (1 - 0.14 * j), 0.9 + 0.05 * j, C.bounce), delay: delay + 0.05 * j });
  }
};

const measure = () => {
  const group = groupRef.current;
  if (!group) return;
  widths.current = chipRefs.current.map(el => el?.offsetWidth ?? 0);
  const chipH = chipRefs.current[0]?.offsetHeight ?? 0;
  const maxW = Math.max(0, ...widths.current);
  group.style.setProperty('--jr-pad-x', `${Math.ceil((maxW * swell * 1.3) / 2 + barge) + 2}px`);
  group.style.setProperty('--jr-pad-y', `${Math.ceil((chipH * swell) / 2) + 2}px`);
};
useLayoutEffect(() => {
  const settle = () => {
    measure();
    apply(applied.current, true);
  };
  settle();
  const observer = new ResizeObserver(settle);
  if (groupRef.current) observer.observe(groupRef.current);
  document.fonts?.ready.then(settle);
  return () => observer.disconnect();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [itemsKey, size, gap, swell, barge, shrink]);
useEffect(() => {
  if (applied.current === at) return;
  applied.current = at;
  apply(at, true);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [at]);
useEffect(
  () => () =>
    mvs.current.forEach(mv => {
      mv.x.destroy();
      mv.sx.destroy();
      mv.sy.destroy();
    }),
  []
);

const commit = (i, instant) => {
  if (disabled || i === at || !list[i] || list[i].disabled) return;
  applied.current = i;
  apply(i, instant);
  if (value === undefined) setInner(list[i].value);
  onChange?.(list[i].value, i);
};
const stepFrom = (i, dir) => {
  const n = list.length;
  let j = i;
  for (let tries = 0; tries < n; tries++) {
    j = (j + dir + n) % n;
    if (!list[j].disabled) return j;
  }
  return i;
};
const onKeyDown = (e, i) => {
  let next = null;
  if (e.key === 'ArrowRight' || e.key === 'ArrowDown') next = stepFrom(i, 1);
  else if (e.key === 'ArrowLeft' || e.key === 'ArrowUp') next = stepFrom(i, -1);
  else if (e.key === 'Home') next = stepFrom(-1, 1);
  else if (e.key === 'End') next = stepFrom(list.length, -1);
  else if (e.key === ' ' || e.key === 'Enter') next = i;
  if (next === null) return;
  e.preventDefault();
  commit(next, true);
  chipRefs.current[next]?.focus();
};

return (
  <div
    ref={groupRef}
```

```
      role="radiogroup"
      aria-label={ariaLabel}
      data-disabled={disabled ? '' : undefined}
      className={`jelly-radio${className ? ` ${className}` : ''}`}
      style={{
        '--jr-chip': chipColor,
        '--jr-active': activeColor,
        '--jr-text': textColor,
        '--jr-active-text': activeTextColor,
        '--jr-gap': `${gap}px`,
        '--jr-radius': `${radius}px`,
        '--jr-h': `${h}px`,
        '--jr-font': `${font}px`,
        '--jr-px': `${px}px`
      }}
    >
      {list.map((it, i) => (
        <Chip
          key={it.value}
          mv={mvFor(i)}
          ref={el => {
            chipRefs.current[i] = el;
          }}
          type="button"
          role="radio"
          aria-checked={i === at}
          tabIndex={i === at ? 0 : -1}
          disabled={disabled || !!it.disabled}
          className="jelly-radio__chip"
          data-on={i === at ? 'true' : 'false'}
          onClick={e => commit(i, e.detail === 0)}
          onKeyDown={e => onKeyDown(e, i)}
        >
          <span className="jelly-radio__skin">
            {it.icon ? <span className="jelly-radio__icon">{it.icon}</span> : null}
            <span className="jelly-radio__label">{it.label}</span>
          </span>
        </Chip>
      ))}
    </div>
  );
}

```

Component CSS
```css
.jelly-radio {
  --jr-chip: #27272a;
  --jr-active: #f5f5f5;
  --jr-text: #f5f5f5;
  --jr-active-text: #18181b;
  --jr-gap: 8px;
  --jr-radius: 18px;
  --jr-h: 36px;
  --jr-font: 13px;
  --jr-px: 16px;
  --jr-pad-x: 16px;
  --jr-pad-y: 6px;
  --jr-ease-out: cubic-bezier(0.23, 1, 0.32, 1);

  display: inline-flex;
  align-items: center;
  gap: var(--jr-gap);
  padding: var(--jr-pad-y) var(--jr-pad-x);
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

.jelly-radio[data-disabled] {
  opacity: 0.5;
  pointer-events: none;
}
```

```
.jelly-radio__chip {
  position: relative;
  margin: 0;
  padding: 0;
  border: 0;
  background: none;
  color: inherit;
  font: inherit;
  cursor: pointer;
  outline: none;
  transform-origin: 50% 50%;
  touch-action: manipulation;
  -webkit-tap-highlight-color: transparent;
}

.jelly-radio__chip[data-on='true'] {
  cursor: default;
}

.jelly-radio:not([data-disabled]) .jelly-radio__chip:disabled {
  cursor: default;
  opacity: 0.4;
}

.jelly-radio__skin {
  position: relative;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.4em;
  height: var(--jr-h);
  padding: 0 var(--jr-px);
  border-radius: var(--jr-radius);
  overflow: hidden;
  background: var(--jr-chip);
  color: var(--jr-text);
  font-size: var(--jr-font);
  font-weight: 500;
  line-height: 1;
  white-space: nowrap;
  transition:
    transform 160ms var(--jr-ease-out),
    background-color 200ms ease,
    color 200ms ease;
}

.jelly-radio__skin::before {
  content: '';
  position: absolute;
  inset: 0;
  background: var(--jr-text);
  opacity: 0;
  pointer-events: none;
  transition: opacity 160ms ease;
}

@media (hover: hover) and (pointer: fine) {
  .jelly-radio__chip[data-on='false']:not(:disabled):hover .jelly-radio__skin::before {
    opacity: 0.11;
  }
}

.jelly-radio__chip:active .jelly-radio__skin {
  transform: scale(0.97);
}

.jelly-radio__chip[data-on='true'] .jelly-radio__skin {
  background: var(--jr-active);
  color: var(--jr-active-text);
}

.jelly-radio__icon {
  display: inline-flex;
```

```
}

@media (prefers-reduced-motion: reduce) {
  .jelly-radio__chip:active .jelly-radio__skin {
    transform: none;
  }
}

```

Integration Instructions
1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

More from React Bits
The full library index, including everything reactbits.dev offers, is at https://reactbits.dev/llms.txt — fetch it if this component is not the right fit or the project needs more pieces.

Top Logo Text Animation (it will stay at the top header and then there will be the menu buttons)-

Integrate the <WarpText /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: WarpText
Variant: JavaScript + CSS
Dependencies: ogl

Usage Example
```jsx
import WarpText from './WarpText';

<WarpText
  text="Bend the moment"
  warpStrength={0.08}
  warpScale={1.7}
  speed={0.55}
  lens="bulge"
  pointerInfluence={0.42}
  pointerStrength={0.38}
  refraction={0.018}
  chromatic={0.35}
  idleDrift={0.2}
  ripple
  clickRipple
  intro
  fontSize="clamp(3rem, 10vw, 9rem)"
  fontWeight={800}
  style={{ height: '320px' }}
  color=null
  letterSpacing="-0.06em"
/>
```

Props
Prop	Type	Default	Description
text	string	"Bend the moment"	The headline text rendered into the WebGL texture. Use \n for line breaks.
color	string	"currentColor"	Text fill color before the glass refraction pass. Follows the surrounding text color by default.
warpStrength	number	0.08	Amount of ambient glass distortion at rest.
warpScale	number	1.7	Size of the moving distortion cells.

speed	number	0.55	Speed of the ambient undulation.
lens	"bulge"	"pinch"	"swirl"	"bulge"	How the cursor bends the text: magnify it, pull it inward or twist it around the pointer.
pointerInfluence	number	0.42	Radius of the cursor lensing area.
pointerStrength	number	0.38	Strength of cursor-driven bending and magnification.
idleDrift	number	0.2	How strongly the lens keeps wandering over the text while the cursor is away, from 0 (it fades out) to 1 (a full-strength lens roaming across the words).
chromatic	number	0	Color fringing inside the lens and click ripples, like light splitting through real glass. 0 is off, 1 is strong.
refraction	number	0.018	Subtle RGB channel split for the glass edge.
ripple	boolean	true	Adds a soft ripple to the pointer lens.
clickRipple	boolean	true	Clicking sends a shockwave ring through the text that bends the letters as it passes and fades out.
intro	boolean	true	Fades the text in from a strong warp that settles once it first comes into view.
fontSize	string	number	"clamp(3rem, 10vw, 9rem)"	Canvas raster font size. Numbers are pixels. The text shrinks to fit the container.
fontWeight	string	number	800	Font weight used when rasterising the text texture.
fontFamily	string	"inherit"	Font family used when rasterising the text texture.
letterSpacing	string	number	"-0.06em"	Tracking applied while drawing the text into the texture.
lineHeight	string	number	0.9	Line height for multi-line text.
className	string	""	Optional class name for the root element.
style	React.CSSProperties	undefined	Optional inline styles for sizing or layout.

Full Component Source
```jsx
'use client';

import { useEffect, useRef, useState } from 'react';
import { Renderer, Program, Mesh, Triangle, Texture } from 'ogl';
import './WarpText.css';

const LENS_MODES = { bulge: 0, pinch: 1, swirl: 2 };
const INTRO_MS = 1600;
const RIPPLE_LIFE = 1.6;
const MAX_CLICKS = 4;

const vertex = `#version 300 es
in vec2 position;
in vec2 uv;
out vec2 vUv;
void main() {
  vUv = uv;
  gl_Position = vec4(position, 0.0, 1.0);
}
`;

const fragment = `#version 300 es
precision highp float;

uniform sampler2D uTextTexture;
uniform vec2 uResolution;
uniform vec2 uPointer;
uniform float uPointerActive;
uniform float uTime;
uniform float uWarpStrength;
uniform float uWarpScale;
uniform float uSpeed;
uniform float uPointerInfluence;
uniform float uPointerStrength;
uniform float uRefraction;
uniform float uChromatic;
uniform float uRipple;
uniform float uMotion;
uniform float uLens;
uniform float uIntro;
uniform float uReveal;
uniform float uBackdrop;
uniform vec4 uClicks[4];

in vec2 vUv;
out vec4 fragColor;

float hash(vec2 p) {
  p = fract(p * vec2(123.34, 456.21));
  p += dot(p, p + 45.32);
  return fract(p.x * p.y);
}
```

```
float noise(vec2 p) {
  vec2 i = floor(p);
  vec2 f = fract(p);
  vec2 u = f * f * (3.0 - 2.0 * f);

  float a = hash(i);
  float b = hash(i + vec2(1.0, 0.0));
  float c = hash(i + vec2(0.0, 1.0));
  float d = hash(i + vec2(1.0, 1.0));

  return mix(mix(a, b, u.x), mix(c, d, u.x), u.y);
}

float fbm(vec2 p) {
  float value = 0.0;
  float amplitude = 0.5;
  for (int i = 0; i < 4; i++) {
    value += amplitude * noise(p);
    p *= 2.02;
    amplitude *= 0.5;
  }
  return value;
}

vec4 sampleText(vec2 uv) {
  if (uv.x < 0.0 || uv.x > 1.0 || uv.y < 0.0 || uv.y > 1.0) {
    return vec4(0.0);
  }
  return texture(uTextTexture, uv);
}

void main() {
  vec2 uv = vUv;
  float aspect = uResolution.x / max(uResolution.y, 1.0);
  float time = uTime * uSpeed;
  float scale = max(uWarpScale, 0.001);

  vec2 drift = vec2(time * 0.055, -time * 0.045);
  float n1 = fbm(uv * scale * 3.1 + drift);
  float n2 = fbm((uv + 19.17) * scale * 3.4 - drift.yx);
  vec2 ambient = (vec2(n1, n2) - 0.5) * (uWarpStrength * 0.045 * uMotion + uIntro * 0.18);

  vec2 pointerDelta = uv - uPointer;
  vec2 aspectDelta = vec2(pointerDelta.x * aspect, pointerDelta.y);
  float dist = length(aspectDelta);
  float radius = max(uPointerInfluence, 0.001);
  float t = clamp(dist / radius, 0.0, 1.0);
  float lens = smoothstep(radius, 0.0, dist) * uPointerActive;
  float bulge = t * (1.0 - t) * (1.0 - t) * 6.75 * uPointerActive;
  vec2 dir = dist > 0.0001 ? vec2(aspectDelta.x / aspect, aspectDelta.y) / dist : vec2(0.0);

  float rippleWave = sin(dist * 28.0 - time * 4.2) * 0.5 + 0.5;
  float rippleRing = (rippleWave - 0.5) * uRipple;
  vec2 pointerWarp = dir * rippleRing * bulge * uPointerStrength * 0.016;

  if (uLens > 1.5) {
    float twist = (1.0 - t) * (1.0 - t) * uPointerActive * uPointerStrength * 2.4;
    float s = sin(twist);
    float c = cos(twist);
    vec2 turned = vec2(aspectDelta.x * c - aspectDelta.y * s, aspectDelta.x * s + aspectDelta.y * c);
    pointerWarp += vec2(turned.x / aspect, turned.y) - pointerDelta;
  } else {
    float side = uLens > 0.5 ? -1.0 : 1.0;
    pointerWarp -= dir * side * bulge * uPointerStrength * 0.045;
  }

  vec2 shock = vec2(0.0);
  float shockLight = 0.0;
  for (int i = 0; i < 4; i++) {
    vec4 click = uClicks[i];
    if (click.w <= 0.0) continue;
    vec2 clickDelta = uv - click.xy;
    vec2 clickAspect = vec2(clickDelta.x * aspect, clickDelta.y);
    float clickDist = length(clickAspect);
```

```
    float age = click.z;
    float offset = (clickDist - age * 0.9) / (0.06 + age * 0.06);
    float fade = 1.0 - smoothstep(0.0, 1.6, age);
    float energy = exp(-offset * offset) * fade * smoothstep(0.0, 0.08, age) * click.w;
    vec2 clickDir = clickDist > 0.0001 ? vec2(clickAspect.x / aspect, clickAspect.y) / clickDist : vec2(0.0);
    shock -= clickDir * offset * energy * 0.09;
    shockLight += energy;
  }

  vec2 bend = pointerWarp + shock;
  vec2 rest = uv + ambient;
  vec2 splitDir = ambient + bend;
  float splitLen = length(splitDir);
  splitDir = splitLen > 0.00001 ? splitDir / splitLen : vec2(0.7071, 0.7071);
  vec2 split = splitDir * uRefraction * 0.16 * (0.35 + lens * 1.65 + uIntro * 3.0);
  float dispersion = uChromatic * 0.5;

  vec3 ink = vec3(0.0);
  vec3 cover = vec3(0.0);
  vec3 weight = vec3(0.0);
  for (int i = 0; i < 7; i++) {
    float k = 1.0 - float(i) / 3.0;
    vec4 tap = sampleText(rest + bend * (1.0 + dispersion * k) + split * k);
    vec3 band = vec3(max(k, 0.0), 1.0 - abs(k), max(-k, 0.0));
    ink += tap.rgb * band;
    cover += tap.a * band;
    weight += band;
  }
  ink /= weight;
  cover /= weight;
  float a = max(max(cover.r, cover.g), cover.b);
  vec3 color = ink + (a - cover) * uBackdrop;
  color = min(color + (lens + shockLight * 0.5) * cover.g * 0.055, vec3(a));
  fragColor = vec4(color, a) * uReveal;
}
`;

const getFontValue = value => (typeof value === 'number' ? `${value}px` : value);

const smooth = value => value * value * (3 - 2 * value);

const getLuminance = color => {
  const probe = document.createElement('canvas');
  probe.width = 1;
  probe.height = 1;
  const ctx = probe.getContext('2d', { willReadFrequently: true });
  if (!ctx) return 1;
  ctx.fillStyle = color;
  ctx.fillRect(0, 0, 1, 1);
  const [r, g, b] = ctx.getImageData(0, 0, 1, 1).data;
  return (0.2126 * r + 0.7152 * g + 0.0722 * b) / 255;
};

const measureLine = (ctx, line, letterSpacing) => {
  const chars = Array.from(line);
  const textWidth = chars.reduce((width, char) => width + ctx.measureText(char).width, 0);
  return textWidth + Math.max(0, chars.length - 1) * letterSpacing;
};

const drawLine = (ctx, line, x, y, letterSpacing) => {
  const chars = Array.from(line);
  let cursor = x - measureLine(ctx, line, letterSpacing) / 2;

  chars.forEach((char, index) => {
    ctx.fillText(char, cursor, y);
    cursor += ctx.measureText(char).width + (index === chars.length - 1 ? 0 : letterSpacing);
  });
};

const buildTextCanvas = ({ container, width, height, dpr, props }) => {
  const canvas = document.createElement('canvas');
  canvas.width = Math.max(1, Math.floor(width * dpr));
  canvas.height = Math.max(1, Math.floor(height * dpr));
```

```
  const ctx = canvas.getContext('2d');
  if (!ctx) return { canvas, ink: props.color };

  const probe = document.createElement('span');
  probe.textContent = props.text;
  Object.assign(probe.style, {
    position: 'absolute',
    visibility: 'hidden',
    pointerEvents: 'none',
    whiteSpace: 'pre',
    inset: '0 auto auto 0',
    color: props.color,
    fontFamily: props.fontFamily,
    fontSize: getFontValue(props.fontSize),
    fontWeight: String(props.fontWeight),
    letterSpacing: getFontValue(props.letterSpacing),
    lineHeight: typeof props.lineHeight === 'number' ? String(props.lineHeight) : props.lineHeight
  });
  container.appendChild(probe);
  const computed = window.getComputedStyle(probe);
  let fontSizePx = parseFloat(computed.fontSize) || 96;
  const ink = computed.color || props.color;
  const fontFamily = computed.fontFamily || 'sans-serif';
  const fontWeight = computed.fontWeight || String(props.fontWeight);
  let letterSpacing = computed.letterSpacing === 'normal' ? 0 : parseFloat(computed.letterSpacing) || 0;
  let lineHeight = parseFloat(computed.lineHeight);
  if (!Number.isFinite(lineHeight)) {
    lineHeight = fontSizePx * (typeof props.lineHeight === 'number' ? props.lineHeight : 0.92);
  }
  probe.remove();

  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  ctx.clearRect(0, 0, width, height);
  ctx.textAlign = 'left';
  ctx.textBaseline = 'middle';
  ctx.fillStyle = ink;
  ctx.imageSmoothingEnabled = true;
  ctx.imageSmoothingQuality = 'high';

  const lines = String(props.text || '').split('\n');
  const applyFont = () => {
    ctx.font = `${fontWeight} ${fontSizePx}px ${fontFamily}`;
  };
  applyFont();

  const maxWidth = width * 0.86;
  const maxHeight = height * 0.78;
  const widest = Math.max(...lines.map(line => measureLine(ctx, line, letterSpacing)), 1);
  const blockHeight = Math.max(lineHeight * lines.length, 1);
  const fit = Math.min(1, maxWidth / widest, maxHeight / blockHeight);

  if (fit < 1) {
    fontSizePx *= fit;
    letterSpacing *= fit;
    lineHeight *= fit;
    applyFont();
  }

  const startY = height / 2 - (lineHeight * (lines.length - 1)) / 2;
  lines.forEach((line, index) => drawLine(ctx, line, width / 2, startY + index * lineHeight, letterSpacing));

  return { canvas, ink };
};

const syncUniforms = (program, props) => {
  const uniforms = program.uniforms;
  uniforms.uWarpStrength.value = props.warpStrength;
  uniforms.uWarpScale.value = props.warpScale;
  uniforms.uSpeed.value = props.speed;
  uniforms.uPointerInfluence.value = props.pointerInfluence;
  uniforms.uPointerStrength.value = props.pointerStrength;
  uniforms.uRefraction.value = props.refraction;
  uniforms.uChromatic.value = props.chromatic;
  uniforms.uRipple.value = props.ripple ? 1 : 0;
```

```
  uniforms.uLens.value = LENS_MODES[props.lens] ?? 0;
};

const WarpText = ({
  text = 'Bend the moment',
  color = 'currentColor',
  warpStrength = 0.08,
  warpScale = 1.7,
  speed = 0.55,
  pointerInfluence = 0.42,
  pointerStrength = 0.38,
  refraction = 0.018,
  chromatic = 0,
  ripple = true,
  clickRipple = true,
  idleDrift = 0.2,
  lens = 'bulge',
  intro = true,
  fontSize = 'clamp(3rem, 10vw, 9rem)',
  fontWeight = 800,
  fontFamily = 'inherit',
  letterSpacing = '-0.06em',
  lineHeight = 0.9,
  className = '',
  style
}) => {
  const containerRef = useRef(null);
  const propsRef = useRef({
    text,
    color,
    fontSize,
    fontWeight,
    fontFamily,
    letterSpacing,
    lineHeight,
    warpStrength,
    warpScale,
    speed,
    pointerInfluence,
    pointerStrength,
    refraction,
    chromatic,
    ripple,
    clickRipple,
    idleDrift,
    lens,
    intro
  });
  const contextRef = useRef(null);
  const [failed, setFailed] = useState(false);

  useEffect(() => {
    propsRef.current = {
      text,
      color,
      fontSize,
      fontWeight,
      fontFamily,
      letterSpacing,
      lineHeight,
      warpStrength,
      warpScale,
      speed,
      pointerInfluence,
      pointerStrength,
      refraction,
      chromatic,
      ripple,
      clickRipple,
      idleDrift,
      lens,
      intro
    };
```

```
  if (contextRef.current) syncUniforms(contextRef.current.program, propsRef.current);
}, [
  text,
  color,
  fontSize,
  fontWeight,
  fontFamily,
  letterSpacing,
  lineHeight,
  warpStrength,
  warpScale,
  speed,
  pointerInfluence,
  pointerStrength,
  refraction,
  chromatic,
  ripple,
  clickRipple,
  idleDrift,
  lens,
  intro
]);

useEffect(() => {
  contextRef.current?.rasterize();
}, [text, color, fontSize, fontWeight, fontFamily, letterSpacing, lineHeight]);

useEffect(() => {
  const container = containerRef.current;
  if (!container || typeof window === 'undefined') return undefined;

  let renderer;
  let gl;
  let program;
  let geometry;
  let mesh;
  let texture;
  let resizeObserver;
  let intersectionObserver;
  let raf = 0;
  let disposed = false;
  let contextLost = false;
  let visible = true;
  let pageVisible = !document.hidden;
  let reduceMotion = window.matchMedia?.('(prefers-reduced-motion: reduce)').matches ?? false;
  let rasterVersion = 0;
  let textReady = false;
  let introProgress = propsRef.current.intro && !reduceMotion ? 0 : 1;
  let lastFrame = 0;
  let frame = 0;
  let lastInk = '';

  const pointer = { x: 0.5, y: 0.5, tx: 0.5, ty: 0.5, active: 0, activeTarget: 0 };
  const clicks = Array.from({ length: MAX_CLICKS }, () => [0.5, 0.5, 0, 0]);
  let clickSlot = 0;
  const startTime = performance.now();

  try {
    renderer = new Renderer({
      webgl: 2,
      alpha: true,
      premultipliedAlpha: true,
      antialias: true,
      dpr: Math.min(window.devicePixelRatio || 1, 2)
    });
    gl = renderer.gl;
  } catch (error) {
    console.warn('WarpText: WebGL could not be initialized.', error);
    setFailed(true);
    return undefined;
  }

  gl.clearColor(0, 0, 0, 0);
  const canvas = gl.canvas;
```

```
canvas.style.position = 'absolute';
canvas.style.inset = '0';
canvas.style.width = '100%';
canvas.style.height = '100%';
canvas.style.display = 'block';
canvas.setAttribute('aria-hidden', 'true');
container.appendChild(canvas);

texture = new Texture(gl, {
  generateMipmaps: false,
  premultiplyAlpha: true,
  minFilter: gl.LINEAR,
  magFilter: gl.LINEAR,
  wrapS: gl.CLAMP_TO_EDGE,
  wrapT: gl.CLAMP_TO_EDGE
});

geometry = new Triangle(gl);
program = new Program(gl, {
  vertex,
  fragment,
  transparent: true,
  depthTest: false,
  depthWrite: false,
  uniforms: {
    uTextTexture: { value: texture },
    uResolution: { value: new Float32Array([1, 1]) },
    uPointer: { value: new Float32Array([0.5, 0.5]) },
    uPointerActive: { value: 0 },
    uTime: { value: 0 },
    uWarpStrength: { value: propsRef.current.warpStrength },
    uWarpScale: { value: propsRef.current.warpScale },
    uSpeed: { value: propsRef.current.speed },
    uPointerInfluence: { value: propsRef.current.pointerInfluence },
    uPointerStrength: { value: propsRef.current.pointerStrength },
    uRefraction: { value: propsRef.current.refraction },
    uChromatic: { value: propsRef.current.chromatic },
    uRipple: { value: propsRef.current.ripple ? 1 : 0 },
    uMotion: { value: reduceMotion ? 0 : 1 },
    uLens: { value: LENS_MODES[propsRef.current.lens] ?? 0 },
    uIntro: { value: 0 },
    uReveal: { value: 0 },
    uBackdrop: { value: 0 },
    uClicks: { value: clicks }
  }
});
mesh = new Mesh(gl, { geometry, program });

const renderOnce = () => {
  if (disposed || contextLost) return;
  renderer.render({ scene: mesh });
};

const rasterize = async () => {
  const version = ++rasterVersion;
  if (document.fonts?.ready) {
    try {
      await document.fonts.ready;
    } catch (error) {
      void error;
    }
  }
  if (disposed || contextLost || version !== rasterVersion) return;

  const rect = container.getBoundingClientRect();
  if (rect.width <= 0 || rect.height <= 0) return;

  const dpr = Math.min(window.devicePixelRatio || 1, 2);
  lastInk = window.getComputedStyle(container).color;
  const raster = buildTextCanvas({
    container,
    width: rect.width,
    height: rect.height,
    dpr,
```

```
    props: propsRef.current
  });
  texture.image = raster.canvas;
  texture.needsUpdate = true;
  program.uniforms.uBackdrop.value = getLuminance(raster.ink) < 0.5 ? 1 : 0;
  textReady = true;
  renderOnce();
};

const resize = () => {
  if (disposed || contextLost) return;
  const rect = container.getBoundingClientRect();
  if (rect.width <= 0 || rect.height <= 0) return;

  renderer.dpr = Math.min(window.devicePixelRatio || 1, 2);
  renderer.setSize(rect.width, rect.height);
  program.uniforms.uResolution.value[0] = gl.drawingBufferWidth;
  program.uniforms.uResolution.value[1] = gl.drawingBufferHeight;
  rasterize();
};

const onPointerMove = event => {
  if (event.pointerType === 'touch') return;
  const rect = canvas.getBoundingClientRect();
  if (rect.width <= 0 || rect.height <= 0) return;
  pointer.tx = (event.clientX - rect.left) / rect.width;
  pointer.ty = 1 - (event.clientY - rect.top) / rect.height;
  pointer.activeTarget = 1;
};

const onPointerLeave = () => {
  pointer.activeTarget = 0;
};

const onPointerDown = event => {
  if (!propsRef.current.clickRipple) return;
  const rect = canvas.getBoundingClientRect();
  if (rect.width <= 0 || rect.height <= 0) return;
  const click = clicks[clickSlot];
  clickSlot = (clickSlot + 1) % clicks.length;
  click[0] = (event.clientX - rect.left) / rect.width;
  click[1] = 1 - (event.clientY - rect.top) / rect.height;
  click[2] = 0;
  click[3] = reduceMotion ? 0.35 : 1;
};

const onContextLost = event => {
  event.preventDefault();
  contextLost = true;
  if (raf) cancelAnimationFrame(raf);
  raf = 0;
  setFailed(true);
};

const onVisibility = () => {
  pageVisible = !document.hidden;
  if (pageVisible && visible && !raf) raf = requestAnimationFrame(loop);
  if (!pageVisible && raf) {
    cancelAnimationFrame(raf);
    raf = 0;
  }
};

const mediaQuery = window.matchMedia?.('(prefers-reduced-motion: reduce)');
const onReducedMotion = event => {
  reduceMotion = event.matches;
  if (reduceMotion) introProgress = 1;
  program.uniforms.uMotion.value = reduceMotion ? 0 : 1;
  renderOnce();
};

const loop = now => {
  if (disposed || contextLost) return;
```

```
  const elapsed = (now - startTime) * 0.001;
  const delta = lastFrame ? Math.min(now - lastFrame, 50) : 16;
  lastFrame = now;
  frame += 1;

  if (textReady && introProgress < 1) introProgress = Math.min(1, introProgress + delta / INTRO_MS);
  const settle = 1 - introProgress;

  if (textReady && frame % 15 === 0 && window.getComputedStyle(container).color !== lastInk) rasterize();

  const drift = Math.min(Math.max(propsRef.current.idleDrift, 0), 1);
  const engaged = pointer.activeTarget > 0;
  const idleX = 0.5 + Math.sin(elapsed * 0.33) * (0.08 + drift * 0.2);
  const idleY = 0.5 + Math.cos(elapsed * 0.27) * (0.06 + drift * 0.2);
  const follow = 1 - Math.exp(-delta / (engaged ? 130 : 470));

  pointer.x += ((engaged ? pointer.tx : idleX) - pointer.x) * follow;
  pointer.y += ((engaged ? pointer.ty : idleY) - pointer.y) * follow;
  pointer.active += ((engaged ? 1 : drift) - pointer.active) * (1 - Math.exp(-delta / 270));

  clicks.forEach(click => {
    if (click[3] <= 0) return;
    click[2] += delta / 1000;
    if (click[2] > RIPPLE_LIFE) click[3] = 0;
  });

  program.uniforms.uPointer.value[0] = pointer.x;
  program.uniforms.uPointer.value[1] = pointer.y;
  program.uniforms.uPointerActive.value = reduceMotion ? pointer.active * 0.35 : pointer.active;
  program.uniforms.uTime.value = reduceMotion ? 0 : elapsed;
  program.uniforms.uIntro.value = settle * settle * settle;
  program.uniforms.uReveal.value = textReady ? smooth(Math.min(1, introProgress / 0.45)) : 0;

  renderOnce();
  raf = requestAnimationFrame(loop);
};

resizeObserver = new ResizeObserver(resize);
resizeObserver.observe(container);

intersectionObserver = new IntersectionObserver(
  ([entry]) => {
    visible = entry.isIntersecting;
    if (visible && pageVisible && !raf) raf = requestAnimationFrame(loop);
    if (!visible && raf) {
      cancelAnimationFrame(raf);
      raf = 0;
    }
  },
  { threshold: 0 }
);
intersectionObserver.observe(container);

canvas.addEventListener('pointermove', onPointerMove);
canvas.addEventListener('pointerleave', onPointerLeave);
canvas.addEventListener('pointerdown', onPointerDown);
canvas.addEventListener('webglcontextlost', onContextLost, false);
document.addEventListener('visibilitychange', onVisibility);
mediaQuery?.addEventListener('change', onReducedMotion);

syncUniforms(program, propsRef.current);
contextRef.current = { program, rasterize };
resize();
raf = requestAnimationFrame(loop);

return () => {
  disposed = true;
  contextRef.current = null;
  if (raf) cancelAnimationFrame(raf);
  resizeObserver?.disconnect();
  intersectionObserver?.disconnect();
  canvas.removeEventListener('pointermove', onPointerMove);
  canvas.removeEventListener('pointerleave', onPointerLeave);
  canvas.removeEventListener('pointerdown', onPointerDown);
```

```
    canvas.removeEventListener('webglcontextlost', onContextLost);
    document.removeEventListener('visibilitychange', onVisibility);
    mediaQuery?.removeEventListener('change', onReducedMotion);

    if (!contextLost) {
      try {
        if (texture?.texture) gl.deleteTexture(texture.texture);
        geometry?.remove?.();
        program?.remove?.();
        gl.getExtension('WEBGL_lose_context')?.loseContext();
      } catch (error) {
        void error;
      }
    }

    if (canvas.parentNode === container) container.removeChild(canvas);
  };
 }, []);

 return (
  <div ref={containerRef} className={`warp-text ${className}`.trim()} style={style} role="img" aria-label={text}>
   {failed && (
    <span
     className="warp-text__fallback"
     aria-hidden="true"
     style={{
      color,
      fontFamily,
      fontWeight,
      fontSize: getFontValue(fontSize),
      letterSpacing: getFontValue(letterSpacing),
      lineHeight
     }}
    >
     {text}
    </span>
   )}
  </div>
 );
};

export default WarpText;

```

Component CSS
```css
.warp-text {
 position: relative;
 display: block;
 width: 100%;
 min-height: 220px;
 overflow: hidden;
 isolation: isolate;
 border-radius: inherit;
}

.warp-text canvas {
 position: absolute;
 inset: 0;
 width: 100%;
 height: 100%;
 display: block;
}

.warp-text__fallback {
 position: absolute;
 inset: 0;
 display: flex;
 align-items: center;
 justify-content: center;
 padding: 0 7%;
 text-align: center;
 white-space: pre-line;
```

}

```

### Integration Instructions
1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

### More from React Bits
The full library index, including everything reactbits.dev offers, is at https://reactbits.dev/llms.txt — fetch it if this component is not the right fit or the project needs more pieces.

# The Offers will be in this style-

---
title: Bento Grid
date: 2023-11-09
description: Bento grid is a layout used to showcase the features of a product in a simple and elegant way.
author: dillionverma
published: true
---

<ComponentPreview name="bento-demo" />

## Installation

<Tabs defaultValue="cli">

<TabsList>
  <TabsTrigger value="cli">CLI</TabsTrigger>
  <TabsTrigger value="manual">Manual</TabsTrigger>
</TabsList>
<TabsContent value="cli">

```bash
npx shadcn@latest add @magicui/bento-grid
```

</TabsContent>

<TabsContent value="manual">

<Steps>

<Step>Copy and paste the following code into your project.</Step>

```tsx
import { type ComponentPropsWithoutRef, type ReactNode } from "react"
import { ArrowRightIcon } from "@radix-ui/react-icons"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"

interface BentoGridProps extends ComponentPropsWithoutRef<"div"> {
  children: ReactNode
  className?: string
}

interface BentoCardProps extends ComponentPropsWithoutRef<"div"> {
  name: string
  className: string
  background: ReactNode
  Icon: React.ElementType
  description: string
  href: string
  cta: string
}
```
```

```
const BentoGrid = ({ children, className, ...props }: BentoGridProps) => {
  return (
    <div
      className={cn(
        "grid w-full auto-rows-[22rem] grid-cols-3 gap-4",
        className
      )}
      {...props}
    >
      {children}
    </div>
  )
}

const BentoCard = ({
  name,
  className,
  background,
  Icon,
  description,
  href,
  cta,
  ...props
}: BentoCardProps) => (
  <div
    key={name}
    className={cn(
      "group relative col-span-3 flex flex-col justify-between overflow-hidden rounded-xl",
      // light styles
      "bg-background [box-shadow:0_0_0_1px_rgba(0,0,0,.03),0_2px_4px_rgba(0,0,0,.05),0_12px_24px_rgba(0,0,0,.05)]",
      // dark styles
      "dark:bg-background transform-gpu dark:[box-shadow:0_-20px_80px_-20px_#ffffff1f_inset] dark:[border:1px_solid_rgba(255,255,255,.1)]",
      className
    )}
    {...props}
  >
    <div>{background}</div>
    <div className="p-4">
      <div className="pointer-events-none z-10 flex transform-gpu flex-col gap-1 transition-all duration-300 lg:group-hover:-translate-y-10">
        <Icon className="h-12 w-12 origin-left transform-gpu text-neutral-700 transition-all duration-300 ease-in-out group-hover:scale-75" />
        <h3 className="text-xl font-semibold text-neutral-700 dark:text-neutral-300">
          {name}
        </h3>
        <p className="max-w-lg text-neutral-400">{description}</p>
      </div>

      <div
        className={cn(
          "pointer-events-none flex w-full translate-y-0 transform-gpu flex-row items-center transition-all duration-300 group-hover:translate-y-0 group-hover:opacity-100
lg:hidden"
        )}
      >
        <Button
          variant="link"
          asChild
          size="sm"
          className="pointer-events-auto p-0"
        >
          <a href={href}>
            {cta}
            <ArrowRightIcon className="ms-2 h-4 w-4 rtl:rotate-180" />
          </a>
        </Button>
      </div>
    </div>

    <div
      className={cn(
        "pointer-events-none absolute bottom-0 hidden w-full translate-y-10 transform-gpu flex-row items-center p-4 opacity-0 transition-all duration-300
group-hover:translate-y-0 group-hover:opacity-100 lg:flex"
      )}
    >
      <Button
```

```
        variant="link"
        asChild
        size="sm"
        className="pointer-events-auto p-0"
      >
        <a href={href}>
          {cta}
          <ArrowRightIcon className="ms-2 h-4 w-4 rtl:rotate-180" />
        </a>
      </Button>
    </div>

    <div className="pointer-events-none absolute inset-0 transform-gpu transition-all duration-300 group-hover:bg-black/3 group-hover:dark:bg-neutral-800/10" />
  </div>
)

export { BentoCard, BentoGrid }

```

<Step>Update the import paths to match your project setup.</Step>

</Steps>

</TabsContent>

</Tabs>

Examples

<ComponentPreview name="bento-demo-vertical" />

Usage

```tsx showLineNumbers
import { BentoCard, BentoGrid } from "@/components/ui/bento-grid"
```

```tsx showLineNumbers
<BentoGrid>
  <BentoCard />
</BentoGrid>
```